

MusicPlayer

Memoria técnica del proyecto intermodular

Título MusicPlayer: plataforma web de gestión y reproducción musical

Autor/a Francisco Huisa y Giovanni Alejandro

Ciclo Desarrollo de Aplicaciones Web

Centro IES Isidra de Guzmán

Tutor Borja Bergua

Fecha 12 de mayo de 2026

Resumen

MusicPlayer es una aplicación web orientada a la gestión de un catálogo musical y a la reproducción organizada de canciones mediante listas, favoritos e historial. El proyecto se ha construido sobre una arquitectura monolítica generada con JHipster 9.0.0, con backend Spring Boot 4.0.3, API REST, persistencia JPA, migraciones Liquibase y frontend Angular 21. El objetivo principal consiste en ofrecer una base funcional para una plataforma musical autogestionada por una organización, con usuarios autenticados, roles diferenciados y operaciones de catálogo controladas.

La solución integra autenticación mediante JSON Web Tokens, control de acceso por autoridades, entidades de dominio para artistas, álbumes, canciones, géneros, playlists, reproducciones y favoritos, además de vistas Angular para administración, edición y consulta del catálogo. La memoria describe el problema abordado, los requisitos funcionales y no funcionales, el diseño de base de datos, la arquitectura por capas, los diagramas de actividad principales, las tecnologías seleccionadas, las pruebas realizadas y las mejoras futuras. También se incluyen anexos de instalación y uso para facilitar la reproducción del entorno.

Palabras clave: MusicPlayer, Angular, Spring Boot, JHipster, JWT, Liquibase.

Abstract

MusicPlayer is a web application designed to manage a music catalogue and organise playback through playlists, likes and listening history. The project is based on a monolithic architecture generated with JHipster 9.0.0, using a Spring Boot 4.0.3 backend, REST API, JPA persistence, Liquibase migrations and an Angular 21 frontend. Its main purpose is to provide a functional foundation for a self-managed music platform operated by an organisation, with authenticated users, differentiated roles and controlled catalogue operations.

The solution integrates JSON Web Token authentication, authority-based access control, domain entities for artists, albums, songs, genres, playlists, plays and likes, as well as Angular views for administration, editing and catalogue browsing. This report describes the problem statement, functional and non-functional requirements, database design, layered architecture, main activity diagrams, selected technologies, validation tests and future improvements. Installation and user-oriented annexes are also included to make the development environment reproducible.

Keywords: MusicPlayer, Angular, Spring Boot, JHipster, JWT, Liquibase.

Índice general

1. Introducción y objetivos del proyecto	7
1.1. Contexto y motivación	7
1.2. Objetivos generales y específicos	7
1.3. Alcance del trabajo	8
2. Análisis del problema y justificación de la solución	8
2.1. Necesidades detectadas	8
2.2. Requisitos funcionales	9
2.3. Requisitos no funcionales	9
2.4. Casos de uso principales	10
2.5. Riesgos y decisiones de mitigación	11
2.6. Estado del arte y alternativas	11
3. Diseño del sistema	12
3.1. Arquitectura general	12
3.2. Modelo entidad-relación explicado	13
3.2.1. Estructura general del modelo	14
3.2.2. Relaciones clave del diseño	14
3.3. Diagrama de clases explicado	15
3.3.1. Responsabilidad de las clases del dominio	16
3.3.2. Clases técnicas de soporte	17
3.4. Diagramas de actividad	18
3.5. Diseño de flujos técnicos	19
3.5.1. Control de acceso por rol	19
3.5.2. Secuencia completa de creación de canción	20
3.5.3. Componentes backend y capas de servicio	20
4. Desarrollo	21
4.1. Tecnologías y herramientas utilizadas	21
4.2. Backend y API REST	22
4.2.1. Clases backend principales	23
4.2.2. Fragmentos de código explicados	23
4.2.3. Flujo backend de creación y publicación de canciones	26

4.2.4. Gestión de ficheros y streaming	26
4.3. Frontend Angular	27
4.3.1. Clases frontend principales	28
4.3.2. Organización de pantallas y navegación	31
4.3.3. Estado reactivo y sincronización del reproductor	31
4.4. Persistencia y migraciones	32
4.4.1. Migraciones Liquibase clave	32
4.4.2. Correspondencia entre JDL, entidades y tablas	33
4.5. Seguridad e internacionalización	34
4.5.1. Recorrido de una petición JWT	35
4.6. Despliegue y ejecución	35
4.7. Planificación temporal	35
4.8. Estimación económica	36
5. Pruebas realizadas y validación	37
5.1. Estrategia de pruebas	37
5.2. Casos de prueba funcionales	38
5.3. Validación técnica	39
5.3.1. Evidencias automáticas disponibles	39
5.3.2. Riesgos no cubiertos completamente	40
5.4. Resultados por módulo	40
5.5. Matriz de trazabilidad	41
5.6. Criterios de aceptación final	42
6. Conclusiones y mejoras futuras	43
6.1. Conclusiones	43
6.2. Mejoras futuras	43
7. Bibliografía y recursos consultados	44
8. Anexos	44
8.1. Anexo A: Manual de instalación	44
8.1.1. Requisitos previos	44
8.1.2. Puesta en marcha en desarrollo	45
8.1.3. Construcción de producción	45
8.2. Anexo B: Manual de usuario	45

8.2.1. Inicio de sesión	45
8.2.2. Gestión de catálogo	45
8.2.3. Playlists, favoritos y reproducciones	46
8.2.4. Reproducción y letras almacenadas	46
8.3. Anexo C: Recursos REST principales	46
8.4. Anexo D: Glosario	47
8.5. Anexo E: Diccionario de datos	47
8.6. Anexo F: Guía de mantenimiento	48

Índice de figuras

Figura 1. Arquitectura general de MusicPlayer. Fuente: elaboración propia.	13
Figura 2. Modelo entidad-relación del sistema. Fuente: imagen del proyecto adaptada para la memoria técnica.	14
Figura 3. Diagrama de clases simplificado del dominio. Fuente: elaboración propia.	16
Figura 4. Diagrama de actividad del inicio de sesión. Fuente: elaboración propia.	18
Figura 5. Diagrama de actividad de creación de canción. Fuente: elaboración propia.	19
Figura 6. Flujo de control de acceso por rol. Fuente: elaboración propia.	20
Figura 7. Secuencia completa de creación de canción. Fuente: elaboración propia.	20
Figura 8. Componentes backend y capas de servicio. Fuente: elaboración propia.	21
Figura 9. Vista funcional del panel de administración. Fuente: representación visual basada en la interfaz implementada.	27
Figura 10. Vista funcional del listado de canciones y del reproductor. Fuente: representación visual basada en la interfaz implementada.	28
Figura 11. Vista funcional del reproductor persistente y del panel de letras. Fuente: representación visual basada en la interfaz implementada.	30
Figura 12. Pirámide de pruebas del proyecto. Fuente: elaboración propia.	38

Índice de tablas

Tabla 1. Requisitos funcionales	9
---------------------------------	---

Tabla 2. Requisitos no funcionales	10
Tabla 3. Casos de uso principales	10
Tabla 4. Riesgos técnicos y mitigación	11
Tabla 5. Comparativa de alternativas	12
Tabla 6. Entidades visibles en el modelo entidad-relación	15
Tabla 7. Responsabilidades de clases del dominio y soporte	17
Tabla 8. Tecnologías empleadas	21
Tabla 9. Migraciones Liquibase clave	32
Tabla 10. Correspondencia entre modelo lógico y modelo físico	33
Tabla 11. Planificación temporal	36
Tabla 12. Estimación económica	36
Tabla 13. Estrategia de pruebas	38
Tabla 14. Casos de prueba funcionales	38
Tabla 15. Evidencias de validación técnica	39
Tabla 16. Resultados por módulo	40
Tabla 17. Matriz de trazabilidad	41
Tabla 18. Recursos REST principales	46
Tabla 19. Diccionario de datos	47
Tabla 20. Operaciones de mantenimiento	48

Índice de abreviaturas y acrónimos

Tabla A. Abreviaturas utilizadas en la memoria

Acrónimo	Significado
API	Application Programming Interface; interfaz de programación usada por el frontend para comunicarse con el backend.
CRUD	Create, Read, Update, Delete; conjunto básico de operaciones sobre entidades.
DTO	Data Transfer Object; objeto usado para transportar datos entre capas.
JWT	JSON Web Token; token firmado que identifica al usuario autenticado.
JPA	Java Persistence API; especificación de persistencia usada con Hibernate.
SPA	Single Page Application; aplicación web que carga una única página y cambia vistas desde el cliente.
TFG	Trabajo de Fin de Grado o proyecto final equivalente.

1. Introducción y objetivos del proyecto

1.1. Contexto y motivación

El consumo musical actual se apoya en plataformas digitales que permiten acceder a catálogos completos desde el navegador o desde dispositivos móviles. Sin embargo, la mayoría de estas plataformas se ofrecen como servicios cerrados, gestionados por empresas externas y orientados a grandes volúmenes de usuarios. Esta situación deja un espacio para aplicaciones autogestionadas que puedan ser instaladas por una organización pequeña, un centro educativo, un colectivo cultural o un equipo de artistas que necesite publicar y organizar su propio contenido sin depender de una plataforma comercial.

MusicPlayer se plantea como una respuesta académica y técnica a ese escenario. No pretende competir con servicios de escala global, sino demostrar cómo se puede diseñar una aplicación musical completa con tecnologías actuales de desarrollo web: autenticación segura, API REST, persistencia relacional, migraciones reproducibles, interfaz responsive y separación clara entre cliente y servidor.

La motivación formativa del proyecto consiste en integrar competencias de desarrollo frontend, backend, bases de datos, seguridad y pruebas. La memoria técnica se ha redactado con un enfoque explicativo para que el lector pueda comprender por qué se ha elegido cada solución y cómo se relacionan las partes del sistema.

1.2. Objetivos generales y específicos

El objetivo general es desarrollar una plataforma web que permita gestionar un catálogo musical y ofrecer a los usuarios autenticados una experiencia básica de consulta, organización y reproducción visual del contenido. La aplicación debía organizar entidades musicales, controlar permisos por rol y mantener una base de datos reproducible mediante migraciones.

Como objetivos específicos se establecen los siguientes:

- Diseñar un modelo de datos relacional para representar artistas, álbumes, canciones, géneros, playlists, reproducciones y favoritos.
- Implementar una API REST que permita operar con las entidades del dominio de forma paginada, validada y segura.
- Configurar autenticación JWT y autorización basada en roles de usuario.
- Construir una interfaz Angular con pantallas de administración, edición y consulta para los distintos perfiles.

- Integrar reproducción de audio y consulta de letras almacenadas dentro del propio modelo de canción.
- Definir pruebas unitarias, de integración y funcionales que validen los flujos principales.
- Documentar la instalación, el uso básico y los límites actuales de la solución.

1.3. Alcance del trabajo

La versión documentada incluye la generación y personalización de una aplicación JHipster con Angular y Spring Boot. Se implementan entidades de negocio, servicios, repositorios, DTO, mappers, controladores REST, rutas Angular, formularios y listados. También se incluye la gestión estándar de usuarios de JHipster, la seguridad JWT y los roles `ROLE_ADMIN`, `ROLE_USER`, `ROLE_EDITOR` y `ROLE_ARTIST`.

El alcance funcional cubre la gestión de catálogo, la organización de playlists, el registro de reproducciones, los favoritos, la visualización de letras almacenadas y una primera solución de reproducción de audio. En la versión actual del proyecto `implementación actual del backend` la subida y el streaming de archivos ya se encuentran presentes mediante `FileUploadResource` y la carpeta `uploads`, aunque su endurecimiento para producción queda identificado como mejora futura.

El proyecto se presenta como prototipo funcional y base ampliable. Por tanto, se valora especialmente la coherencia del diseño, la reproducibilidad del entorno, la claridad de la arquitectura y la trazabilidad entre requisitos, implementación y pruebas.

2. Análisis del problema y justificación de la solución

2.1. Necesidades detectadas

Una plataforma musical autogestionada necesita resolver tres grupos de necesidades. En primer lugar, necesita gestionar contenido: canciones, artistas, álbumes y géneros deben guardarse con información suficiente para poder ser buscados, ordenados y relacionados. En segundo lugar, debe permitir que los usuarios organicen ese contenido mediante playlists, favoritos e historial. En tercer lugar, debe proteger las operaciones sensibles para que la administración del catálogo no quede expuesta a cualquier usuario autenticado.

La solución elegida se justifica por la combinación de Spring Boot y Angular. Spring Boot proporciona una base robusta para exponer una API REST y aplicar seguridad.

Angular facilita una interfaz modular y escalable. JHipster permite unir ambas tecnologías con una estructura inicial coherente, generando capas, pruebas y migraciones que después se adaptan al dominio musical.

2.2. Requisitos funcionales

La Tabla 1 recoge los requisitos funcionales principales. Se han redactado en términos verificables para facilitar su relación con los casos de prueba.

Tabla 1. Requisitos funcionales

ID	Requisito	Rol principal
RF-01	El sistema debe permitir autenticarse con usuario y contraseña y recibir un token JWT.	Todos
RF-02	El sistema debe permitir registrar usuarios y gestionar la cuenta mediante los mecanismos estándar de JHipster.	Público / usuario
RF-03	El sistema debe permitir crear, consultar, modificar y eliminar canciones con título, duración, URL de archivo, portada, letra y fecha de lanzamiento.	ADMIN, EDITOR, ARTIST
RF-04	El sistema debe permitir gestionar álbumes asociados a artistas y clasificados por tipo: álbum, single, EP o serie de podcast.	ADMIN, EDITOR, ARTIST
RF-05	El sistema debe permitir gestionar artistas con biografía, imagen, país y estado de verificación.	ADMIN, EDITOR
RF-06	El sistema debe permitir gestionar géneros musicales y relacionarlos con álbumes y canciones.	ADMIN, EDITOR
RF-07	El sistema debe permitir crear playlists públicas o privadas y añadir canciones con una posición determinada.	USER
RF-08	El sistema debe registrar reproducciones asociadas a usuario y canción.	USER
RF-09	El sistema debe permitir marcar canciones como favoritas mediante la entidad Like.	USER
RF-10	El sistema debe mostrar dashboards diferenciados para administración, edición y usuario final.	Todos
RF-11	El sistema debe ofrecer una barra de reproductor persistente con controles visuales de reproducción, volumen, progreso, repetición y aleatorio.	Todos
RF-12	El sistema debe permitir reproducir audio mediante el reproductor persistente y consultar letras almacenadas en la información de la canción.	Todos

2.3. Requisitos no funcionales

La Tabla 2 enumera los requisitos no funcionales. Estos requisitos condicionan las decisiones de arquitectura y validación.

Tabla 2. Requisitos no funcionales

ID	Requisito	Criterio de validación
RNF-01	Seguridad	Las rutas de API bajo <code>/api/**</code> requieren autenticación salvo autenticación, registro y recuperación de cuenta.
RNF-02	Control de acceso	Las rutas de administración requieren <code>ROLE_ADMIN</code> y las vistas de edición de catálogo limitan el acceso a roles autorizados.
RNF-03	Mantenibilidad	El proyecto mantiene separación entre dominio, repositorio, servicio, DTO, mapper y controlador REST.
RNF-04	Reproducibilidad	El esquema de base de datos se versiona mediante 16 changelogs Liquibase.
RNF-05	Internacionalización	La interfaz dispone de recursos en español e inglés mediante ngx-translate.
RNF-06	Usabilidad	La interfaz usa dashboards, navegación lateral, formularios validados y paginación.
RNF-07	Observabilidad	El backend incluye Actuator para salud, métricas y configuración, restringiendo endpoints sensibles.
RNF-08	Pruebas	El repositorio incluye 87 pruebas Java y 116 especificaciones frontend TypeScript.

2.4. Casos de uso principales

Los casos de uso resumen las interacciones más importantes entre actores y sistema. La Tabla 3 muestra los casos que sirven de base a los flujos explicados en el diseño.

Tabla 3. Casos de uso principales

Código	Actor	Descripción	Resultado esperado
CU-01	Usuario no autenticado	Iniciar sesión mediante formulario.	El sistema valida credenciales, emite JWT y redirige al dashboard correspondiente.
CU-02	Administrador	Gestionar usuarios, roles y monitorización.	Se accede a las pantallas de administración y a endpoints protegidos.
CU-03	Editor o artista	Crear o modificar contenido del catálogo.	La canción, álbum o artista queda persistido y visible en listados.
CU-04	Usuario final	Crear una playlist y añadir canciones.	La playlist queda asociada al usuario y mantiene el orden de canciones.
CU-05	Usuario final	Marcar canciones como favoritas.	Se crea o actualiza una relación Like entre usuario y canción.
CU-06	Usuario final	Reproducir una canción y consultar su información adicional.	El reproductor solicita el audio al backend y la vista muestra la letra almacenada si existe.

2.5. Riesgos y decisiones de mitigación

El principal riesgo técnico del proyecto es que el uso de un generador como JHipster produzca documentación automática poco explicada. Para mitigarlo, esta memoria separa claramente la descripción del generador de las decisiones propias del dominio. Otro riesgo es la gestión directa de ficheros multimedia en disco, mitigada mediante validación de tipos, publicación controlada del directorio `uploads` y comprobaciones de seguridad en el endpoint de streaming. También existe riesgo de discrepancia entre entornos de base de datos; por ello se utiliza Liquibase como mecanismo de evolución y validación del esquema.

Tabla 4. Riesgos técnicos y mitigación

Riesgo	Impacto	Mitigación aplicada o propuesta
Gestión local de audio e imágenes	Un fichero malicioso o una ruta manipulada podría comprometer el servidor.	Validación de tipo MIME, normalización de rutas y publicación controlada de <code>uploads</code> .
Desfase entre documentación y código	La memoria podría prometer funcionalidades no implementadas.	Revisión de JDL, rutas, servicios y componentes antes de redactar la versión final.
Exposición de operaciones sensibles	Usuarios no autorizados podrían modificar catálogo o usuarios.	JWT, reglas de Spring Security, rutas protegidas y directivas de autoridad en Angular.
Crecimiento del modelo de datos	Relaciones complejas pueden dificultar consultas y mantenimiento.	DTO, MapStruct, repositorios separados y modelo explicado con diagramas.
Migraciones inconsistentes	Un entorno podría tener un esquema distinto al esperado.	Uso de Liquibase y changelogs versionados en el repositorio.
Acoplamiento excesivo frontend-backend	Cambios en entidades podrían romper pantallas.	Comunicación por API REST y modelos TypeScript generados por entidad.

2.6. Estado del arte y alternativas

Antes de justificar MusicPlayer conviene situarlo frente a alternativas existentes. Las grandes plataformas comerciales ofrecen una experiencia muy completa, pero no están diseñadas para que una organización pequeña instale su propia instancia y controle usuarios, catálogo y permisos. Las alternativas de código abierto, por su parte, suelen centrarse en bibliotecas personales o servidores musicales ya consolidados, pero no siempre ofrecen una arquitectura didáctica que combine backend Java, frontend moderno, seguridad JWT y generación reproducible de entidades.

La Tabla 5 compara MusicPlayer con varias categorías de soluciones. El objetivo de esta comparación no es afirmar superioridad funcional frente a plataformas maduras, sino

explicar el hueco académico y técnico que cubre el proyecto: una aplicación autogestionable, comprensible y ampliable como ejercicio completo de desarrollo web.

Tabla 5. Comparativa de alternativas

Solución	Ventaja principal	Limitación para el caso del proyecto	Aporte de MusicPlayer
Spotify / Apple Music	Catálogo masivo, experiencia de usuario muy pulida y recomendaciones avanzadas.	No son autogestionables y no permiten controlar una instancia propia.	Modelo instalable y adaptable a una organización concreta.
YouTube Music	Gran variedad de contenido y fuerte integración con el ecosistema Google.	Dependencia total de una plataforma externa y catálogo no controlado por la organización.	Control del catálogo y de la base de usuarios.
SoundCloud	Facilita la publicación de artistas independientes.	Sistema cerrado y menor control administrativo por parte de una entidad externa.	Roles diferenciados y administración interna.
Navidrome / Ampache	Servidores musicales de código abierto para bibliotecas propias.	Objetivos distintos y menor integración didáctica con una arquitectura Spring + Angular.	Proyecto académico con API REST, DTO, pruebas y frontend integrado.
Aplicación desarrollada desde cero	Control completo del código desde el primer archivo.	Mayor tiempo de configuración de seguridad, build, migraciones y pruebas.	JHipster acelera la base técnica y permite centrarse en el dominio musical.

La decisión de usar JHipster tiene sentido en este contexto porque reduce el tiempo dedicado a infraestructura repetitiva. No obstante, se ha evitado presentar el resultado como una simple salida del generador. La memoria explica qué elementos se han aprovechado, cómo se han adaptado al dominio musical y qué partes requieren evolución para alcanzar una plataforma de streaming plenamente productiva.

3. Diseño del sistema

3.1. Arquitectura general

MusicPlayer sigue una arquitectura de aplicación web monolítica con cliente Angular y servidor Spring Boot. Aunque ambos forman parte del mismo proyecto Maven/JHipster, se comunican mediante HTTP y JSON. La Figura 1 muestra esta estructura general: el navegador ejecuta la SPA Angular, el backend expone la API REST, la base de datos almacena el estado persistente y el subsistema de ficheros sirve imágenes y audio desde la carpeta `uploads`.

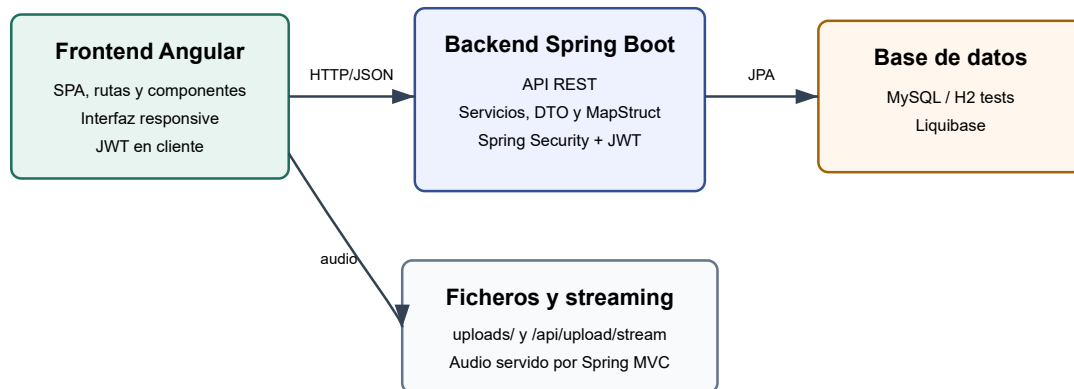


Figura 1. Arquitectura general de MusicPlayer. Fuente: elaboración propia.

La decisión de usar esta arquitectura se justifica por el alcance del proyecto. Un diseño de microservicios no aportaría beneficios claros para un prototipo académico y aumentaría la complejidad operativa. En cambio, el monolito modular permite separar capas dentro de un único despliegue: controlador REST, servicio, repositorio, entidad, mapper y configuración MVC para recursos estáticos.

3.2. Modelo entidad-relación explicado

El modelo entidad-relación se ha sustituido por la imagen de trabajo utilizada durante el desarrollo y almacenada en la carpeta de ejemplos del proyecto. Esta versión resulta más fiel al esquema realmente manejado por el equipo y evita inconsistencias de flechas o cardinalidades que aparecían en el diagrama anterior.

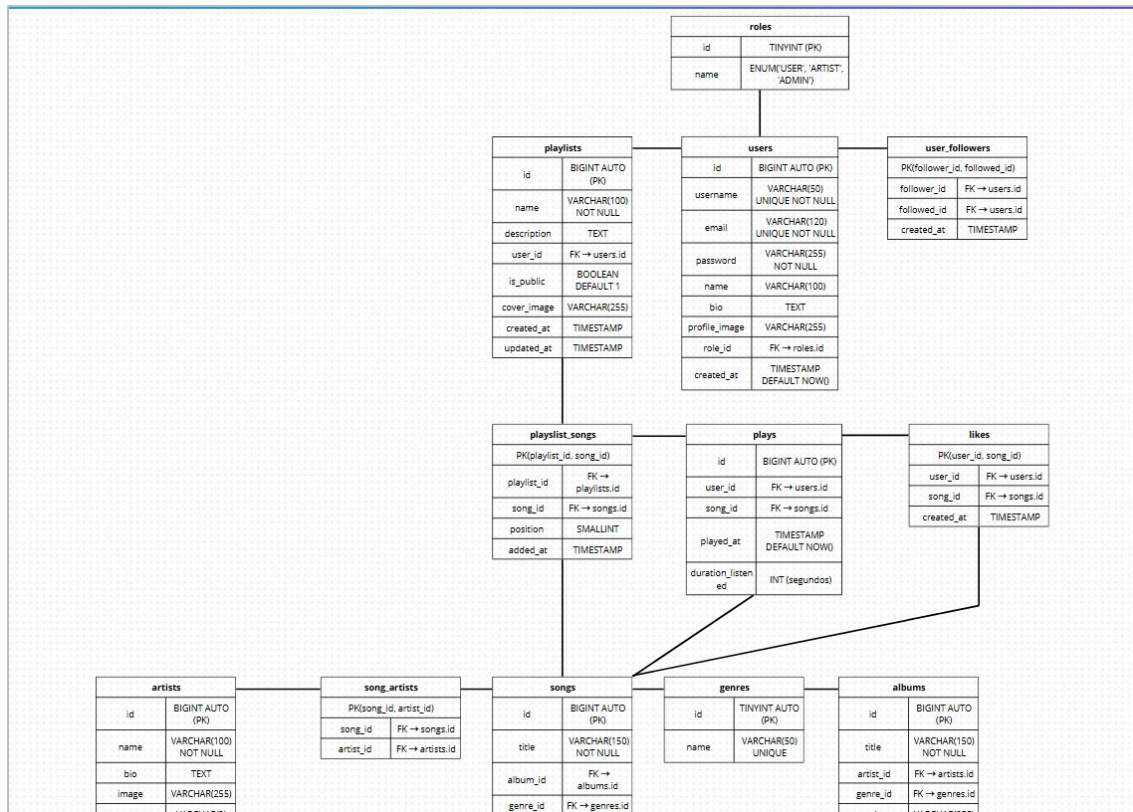


Figura 2. Modelo entidad-relación del sistema. Fuente: imagen del proyecto adaptada para la memoria técnica.

3.2.1. Estructura general del modelo

La figura organiza el sistema en tres bloques funcionales. El primero es el bloque de identidad y permisos, formado por `roles`, `users` y `user_followers`. Aquí se define quién accede a la plataforma, con qué perfil lo hace y cómo se representan relaciones sociales entre usuarios.

El segundo bloque recoge la interacción de una persona usuaria con el catálogo musical. En este nivel aparecen `playlists`, `playlist_songs`, `plays` y `likes`. Estas entidades no describen la canción en sí misma, sino la actividad que se genera alrededor de ella: listas personalizadas, orden de reproducción, historial de escucha y favoritos.

El tercer bloque corresponde al catálogo principal. Está compuesto por `artists`, `song_artists`, `songs`, `genres` y `albums`. La entidad `songs` ocupa una posición central porque conecta la autoría musical, la clasificación por género, la pertenencia a álbum y la reutilización posterior en favoritos, reproducciones y playlists.

3.2.2. Relaciones clave del diseño

La relación entre `users` y `roles` permite distinguir perfiles administrativos, editoriales, artísticos y de consumo. La tabla `user_followers` introduce una relación autorreferente

entre usuarios, útil para ampliar la dimensión social de la plataforma sin duplicar la información principal de la cuenta.

Las playlists se modelan mediante una cabecera en `playlists` y una tabla intermedia `playlist_songs`, que añade el atributo `position`. Esta decisión es importante porque el orden dentro de una lista forma parte del comportamiento funcional de la aplicación y no puede resolverse con una relación muchos a muchos simple.

La autoría musical se divide entre un artista principal y colaboraciones. La tabla `song_artists` evita duplicar datos y permite que una canción mantenga varias relaciones de autoría. Del mismo modo, `albums` y `genres` se conectan con `songs` para ordenar el catálogo y hacer posibles búsquedas, filtros y agrupaciones coherentes.

Tabla 6. Entidades visibles en el modelo entidad-relación

Entidad	Papel dentro del sistema
<code>roles</code>	Define los perfiles de acceso y las capacidades generales del sistema.
<code>users</code>	Representa las cuentas autenticables de la plataforma.
<code>user_followers</code>	Modela el seguimiento entre usuarios.
<code>playlists</code>	Cabecera de listas personales o públicas.
<code>playlist_songs</code>	Resuelve la asociación entre playlist y canción conservando el orden.
<code>plays</code>	Registra reproducciones e información temporal asociada.
<code>likes</code>	Persistencia de canciones marcadas como favoritas.
<code>artists</code>	Información autoral y de gestión de intérpretes.
<code>song_artists</code>	Relaciona canciones con artistas colaboradores.
<code>songs</code>	Entidad central del catálogo y de la reproducción.
<code>genres</code>	Clasificación temática del catálogo.
<code>albums</code>	Agrupación editorial de canciones.

La lectura conjunta de la figura y la tabla anterior permite justificar que el modelo relacional no es un apéndice del proyecto, sino la base que hace posible catálogo, reproducción, favoritos, seguimiento y organización musical. Por ello, esta representación se mantiene en el cuerpo principal de la memoria y no en los anexos.

3.3. Diagrama de clases explicado

La Figura 3 simplifica el diagrama de clases del dominio. No se representan todos los métodos generados por JHipster porque no aportarían comprensión al lector. En su lugar, se muestran atributos relevantes y cardinalidades. Esta decisión responde a la

recomendación del tutor: el diagrama debe explicar el diseño, no convertirse en un volcado automático de clases.

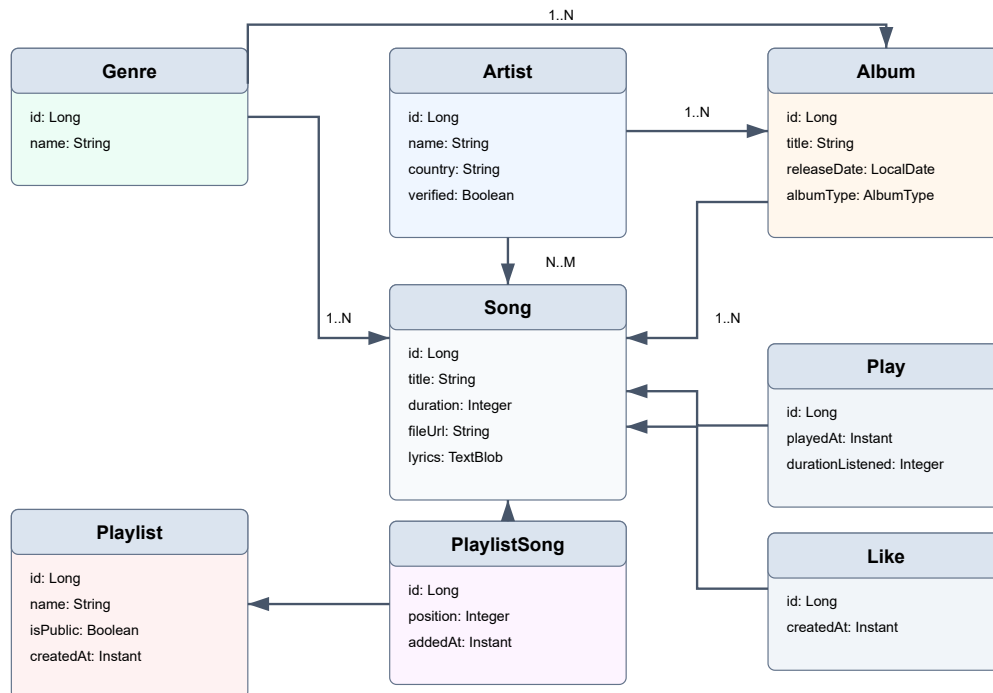


Figura 3. Diagrama de clases simplificado del dominio. Fuente: elaboración propia.

En la versión actual del proyecto analizada destacan varias clases por su papel estructural. `Song` concentra la mayor parte del dominio musical, ya que enlaza con `Album`, `Genre`, `Artist`, colaboradores y datos propios como `fileUrl`, `lyrics` o `active`. `Artist` añade la relación con `User`, lo que permite que una canción pueda asociarse automáticamente al artista autenticado. `PlaylistSong` se comporta como una clase de asociación enriquecida y resuelve el problema del orden dentro de una lista.

El uso de DTO y MapStruct evita exponer directamente las entidades JPA en la API. Cada recurso REST recibe y devuelve objetos de transferencia, mientras que los mappers convierten entre DTO y entidad. Esto mejora la mantenibilidad porque permite cambiar detalles internos del modelo sin modificar necesariamente el contrato público. En este proyecto, esta separación también sirve para aplicar reglas de negocio en servidor, por ejemplo al impedir que el cliente asigne libremente el artista propietario de una canción.

3.3.1. Responsabilidad de las clases del dominio

La explicación del diagrama de clases no debe quedarse en una enumeración de nombres. Cada clase existe para resolver una necesidad concreta. `Song` actúa como agregado central del catálogo y coordina información editorial, relaciones con otras entidades y visibilidad pública. `Album` agrupa canciones bajo una publicación reconocible, mientras que `Genre` ofrece clasificación temática y facilita filtros. `Artist`

aporta contexto autoral, biografía, imagen y vinculación con el usuario autenticado que gestiona el contenido.

Las clases `Playlist`, `PlaylistSong`, `Like` y `Play` no definen el catálogo, sino el comportamiento del usuario sobre él. Esta separación entre catálogo e interacción evita cargar la entidad canción con datos que pertenecen a la experiencia personalizada. Gracias a ello, el sistema puede escalar conceptualmente: una canción es la misma para todos, pero cada usuario mantiene su propio historial, sus favoritos y sus listas.

Tabla 7. Responsabilidades de clases del dominio y soporte

Clase o componente	Papel técnico	Valor dentro del proyecto
Song	Entidad principal del catálogo musical.	Concentra título, duración, fichero, letra, fecha, estado y relaciones musicales.
Artist	Entidad asociada a la autoría y gestión musical.	Permite ligar catálogo con identidad de usuario y propiedad del contenido.
Album	Agrupador editorial de canciones.	Aporta contexto de publicación y mejora la navegación del catálogo.
PlaylistSong	Clase de asociación con atributos propios.	Conserva el orden de reproducción y la fecha de inserción.
SongDTO	Contrato de transporte entre API y cliente.	Evita exponer entidades JPA completas y facilita validación.
SongMapper	Conversión entre DTO y entidad.	Centraliza el mapeo y protege campos sensibles como el artista propietario.
SongServiceImpl	Lógica de negocio de canciones.	Aplica reglas reales de publicación, propiedad y visibilidad.
SongRepository	Acceso a datos y consultas especializadas.	Resuelve filtros públicos, búsquedas y listados paginados.

3.3.2. Clases técnicas de soporte

Además del dominio puro, la aplicación depende de varias clases técnicas que unen infraestructura y negocio. `DomainUserDetailsService` traduce un usuario persistido a un objeto comprensible para Spring Security. `AuthenticateController` crea el token JWT que viaja en cada petición autenticada. `FileUploadResource` abstrae la entrada y salida de ficheros. Por su parte, `PlayerService` en Angular convierte la selección de una canción en un estado reactivo y en una reproducción real sobre la API `Audio` del navegador.

Estas clases no son accesorias. Son las responsables de que el modelo musical pueda utilizarse en una aplicación real. Un catálogo sin autenticación permitiría accesos indebidos; una canción sin controlador de subida no podría asociarse a un audio real; un

listado sin servicio de reproducción no ofrecería la experiencia mínima esperable. Por eso la memoria incorpora su explicación y no se limita a mostrar entidades.

3.4. Diagramas de actividad

El inicio de sesión es el primer flujo crítico. Como se muestra en la Figura 4, el usuario introduce credenciales, el backend valida la autenticación y, si es correcta, el cliente guarda el JWT y redirige según el rol. Si las credenciales son incorrectas, el formulario muestra un error sin crear sesión.

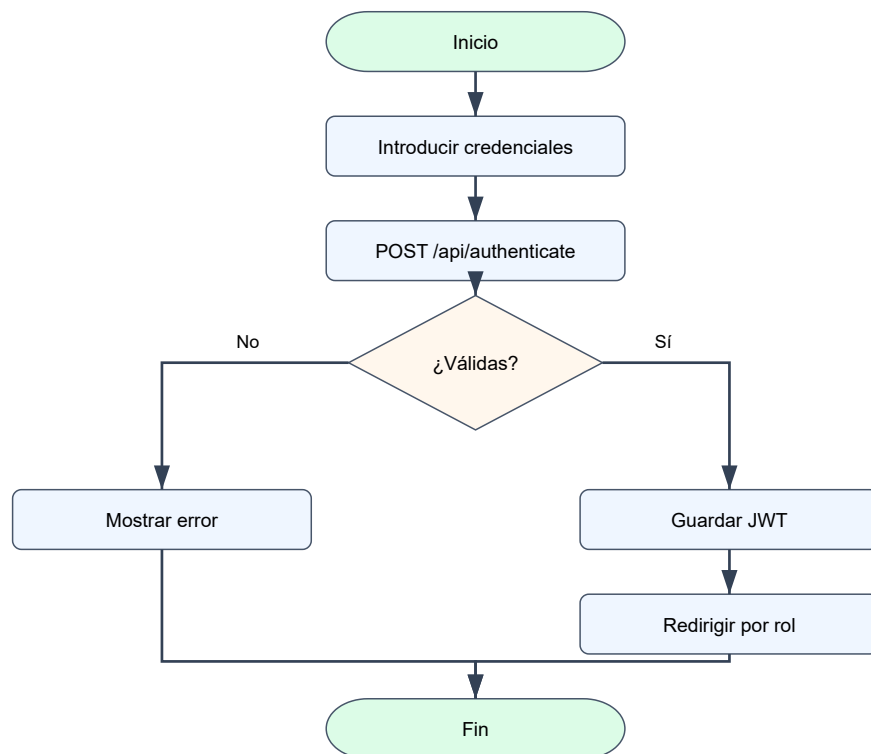


Figura 4. Diagrama de actividad del inicio de sesión. Fuente: elaboración propia.

El segundo flujo crítico corresponde a la creación de canciones, representado en la Figura 5. La validación se realiza en dos niveles: el formulario Angular comprueba campos obligatorios antes de enviar la petición y el backend vuelve a validar el DTO mediante Bean Validation y seguridad por rol. Esta doble validación evita depender únicamente del cliente.

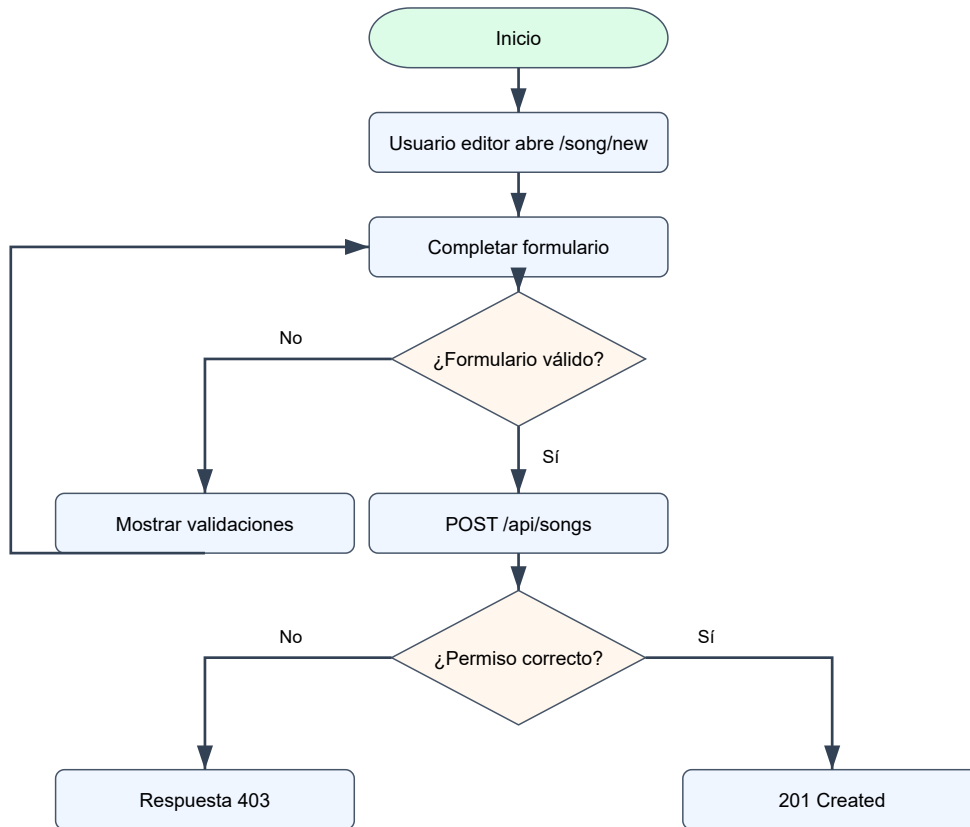


Figura 5. Diagrama de actividad de creación de canción. Fuente: elaboración propia.

3.5. Diseño de flujos técnicos

Además de los diagramas de actividad generales, interesa documentar algunos recorridos técnicos concretos porque son los que conectan frontend, seguridad, lógica de negocio y persistencia. Los tres diagramas siguientes se han incorporado para reforzar la explicación del diseño y corregir la carencia de detalle señalada en la revisión.

3.5.1. Control de acceso por rol

El control de acceso se resuelve en dos planos. En el cliente, los guards y la navegación protegen la experiencia de uso; en el servidor, Spring Security aplica la autorización efectiva. La Figura 6 resume el recorrido que sigue una solicitud desde que la persona usuaria intenta abrir una ruta protegida hasta que el sistema permite el acceso, exige autenticación o devuelve acceso denegado.

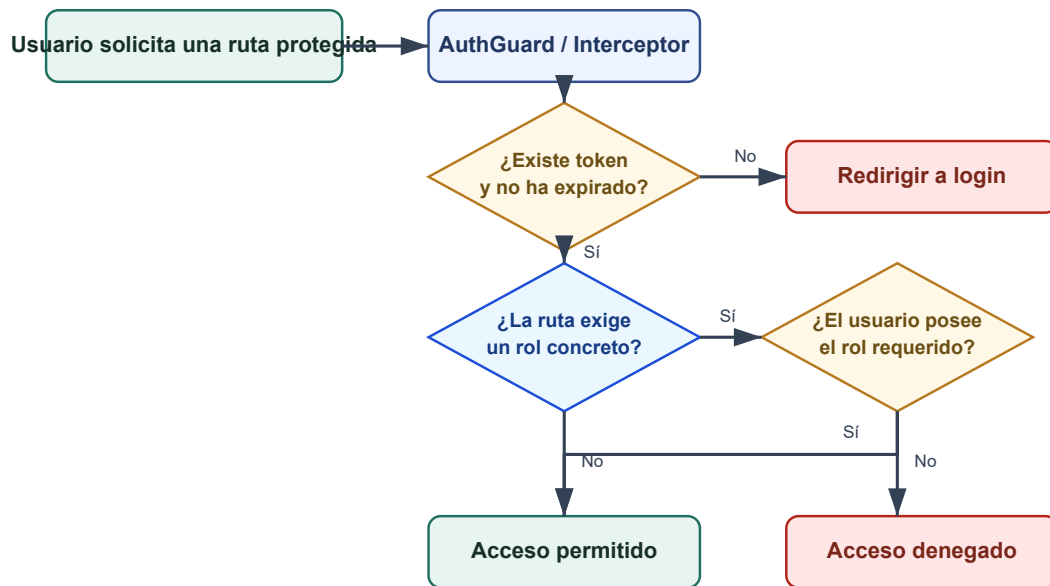


Figura 6. Flujo de control de acceso por rol. Fuente: elaboración propia.

3.5.2. Secuencia completa de creación de canción

La creación de canciones concentra validación de permisos, resolución del artista propietario y persistencia de relaciones musicales. La Figura 7 expone esa secuencia de extremo a extremo, desde el formulario Angular hasta la escritura final en repositorio y base de datos.

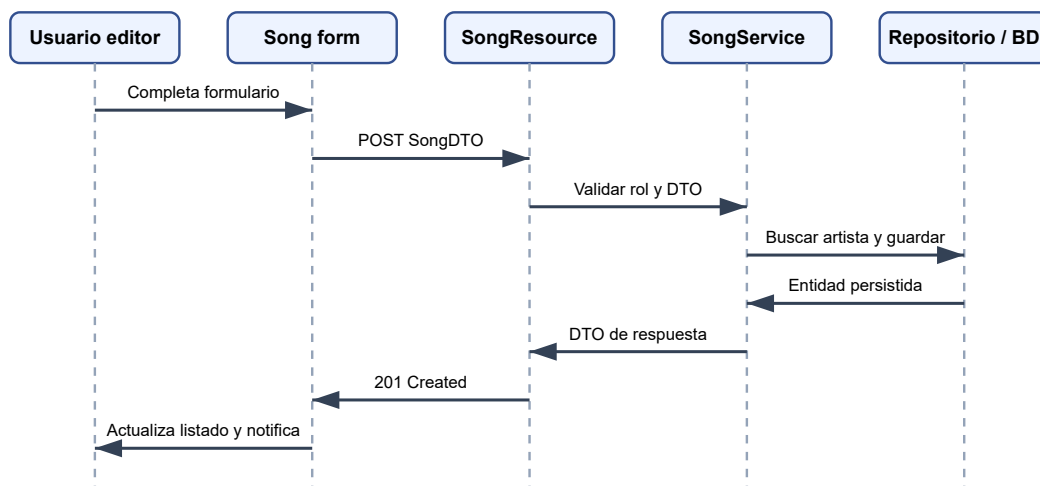


Figura 7. Secuencia completa de creación de canción. Fuente: elaboración propia.

3.5.3. Componentes backend y capas de servicio

El backend se apoya en una separación clara entre recursos REST, servicios, mappers, repositorios e infraestructura. La Figura 8 resume esa organización para que el lector

pueda relacionar las clases explicadas en el desarrollo con una vista de conjunto más fácil de seguir.

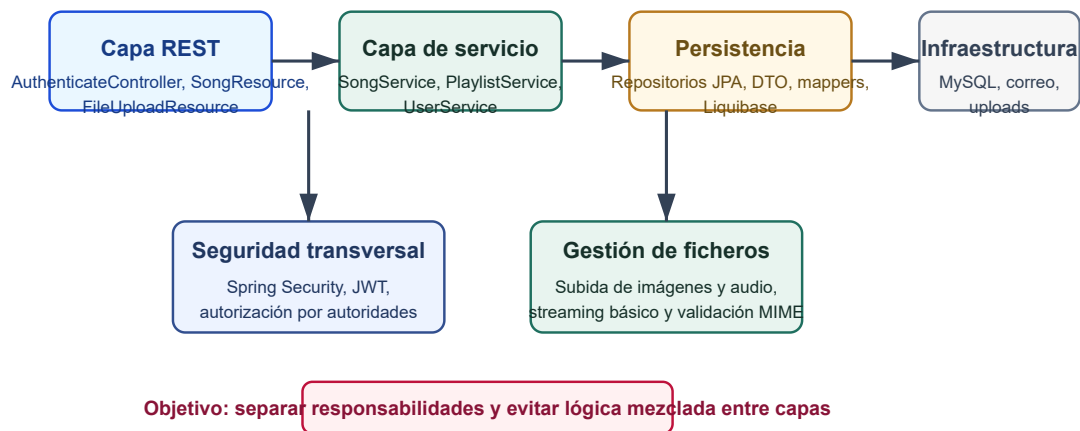


Figura 8. Componentes backend y capas de servicio. Fuente: elaboración propia.

Con estos tres recorridos se completa la parte de diseño que faltaba explicar con mayor claridad: autorización, secuencia operativa y distribución de responsabilidades técnicas. Esta ampliación responde directamente a la necesidad de reforzar los diagramas sin alterar el núcleo del contenido ya validado.

4. Desarrollo

4.1. Tecnologías y herramientas utilizadas

La Tabla 8 resume las herramientas empleadas. Se han incluido únicamente las tecnologías con impacto directo sobre la arquitectura o el proceso de desarrollo.

Tabla 8. Tecnologías empleadas

Tecnología	Versión / uso	Justificación
JHipster	9.0.0	Generación de base Spring Boot + Angular, seguridad JWT, estructura de capas y Liquibase.
Java	21	Lenguaje del backend y versión configurada en Maven.
Spring Boot	4.0.3	Servidor REST, inyección de dependencias, seguridad, validación, Actuator y mail.
Angular	21.2.2	Frontend SPA, rutas, componentes standalone y señales reactivas.
TypeScript	5.9.3	Tipado del frontend y mejora de mantenibilidad.
MySQL	8 / configuración local	Base de datos relacional para entornos de desarrollo y producción.

Tecnología	Versión / uso	Justificación
Liquibase	Maven plugin y changelogs XML	Migraciones reproducibles y versionadas.
MapStruct	1.6.3	Conversión entre entidades y DTO.
Bootstrap / ng-bootstrap	5.3.8 / 20.0.0	Componentes visuales, formularios, dropdowns y paginación.
Vitest y JUnit 5	Pruebas frontend y backend	Validación automática de servicios, componentes y recursos REST.

SweetAlert2. En el frontend se ha utilizado además `SweetAlert2` para confirmaciones y mensajes de interacción. La librería aparece en operaciones de administración de usuarios, edición de artistas, alta de géneros y acciones sobre playlists, mejorando la claridad de los cuadros modales frente a las alertas nativas del navegador.

4.2. Backend y API REST

El backend se encuentra en el paquete `com.musicplayer` y sigue la estructura habitual de JHipster. Las entidades JPA están en `domain`, los repositorios en `repository`, la lógica de aplicación en `service`, los DTO en `service.dto`, los mappers en `service.mapper` y los controladores en `web.rest`. Esta organización permite localizar rápidamente la responsabilidad de cada clase.

La API REST expone 12 recursos principales, incluyendo los recursos de negocio y los recursos de administración de usuarios/autoridades. Las operaciones de lista emplean paginación y devuelven la cabecera `X-Total-Count`, lo que permite al frontend mostrar paginadores sin descargar todos los registros. Las operaciones de creación y actualización aplican validaciones declaradas en los DTO o entidades generadas.

La configuración de seguridad declara la aplicación como stateless: no se almacenan sesiones de servidor, sino que cada petición autenticada incluye un token en la cabecera `Authorization`. Las rutas públicas quedan limitadas a autenticación, registro, activación, recuperación de contraseña, contenido estático y endpoints de salud. El resto de rutas bajo `/api/**` requiere autenticación.

Un aspecto relevante es el uso de roles personalizados. Además de `ROLE_ADMIN` y `ROLE_USER`, se han incorporado `ROLE_EDITOR` y `ROLE_ARTIST`. Estos roles permiten distinguir entre usuarios que consumen contenido y usuarios que pueden gestionar catálogo. La base de datos incluye migraciones para registrar estas autoridades.

4.2.1. Clases backend principales

`SongResource` es el punto de entrada REST para el catálogo musical. Su responsabilidad no se limita a exponer CRUD: también separa vistas públicas, vistas del artista autenticado y vistas de administración mediante rutas como `/api/songs`, `/api/songs/my-songs` y `/api/songs/admin`. Esta distinción hace visible, en la propia API, que no todos los consumidores operan sobre el mismo subconjunto de datos.

`SongServiceImpl` concentra la lógica de aplicación asociada a canciones. Aquí se decide qué artista debe quedar vinculado al registro, qué campos se pueden actualizar, cómo se filtran las canciones públicas y qué ocurre al activar o desactivar un tema. `SongRepository` encapsula consultas más específicas, por ejemplo recuperar solo canciones activas y ya publicables según su fecha de lanzamiento.

`FileUploadResource` y `StaticResourceConfig` son dos clases especialmente importantes en esta versión del proyecto porque materializan una funcionalidad que afecta al uso real de la plataforma: la carga y el servido de ficheros. La primera valida y guarda audio e imágenes; la segunda publica la carpeta configurada en `app.upload.dir` bajo la ruta `/uploads/**`.

4.2.2. Fragmentos de código explicados

El primer fragmento relevante aparece en `SongMapper`:

```
@Mapping(target = "id", ignore = true)

@Mapping(target = "artist", ignore = true)

Song toEntity(SongDTO songDTO);
```

La omisión del campo `artist` no es accidental. El mapper impide que el cliente construya directamente una entidad `Song` con un artista arbitrario. De esta forma, la propiedad del contenido no queda en manos del formulario enviado por el navegador.

Esa decisión se completa en `SongServiceImpl.save`:

```
String login = SecurityUtils.getCurrentUserLogin().orElseThrow(...);

Artist artist =
    artistRepository.findByUserLogin(login).orElseThrow(...);

song.setArtist(artist);

song = songRepository.save(song);
```

Aquí se observa una regla de negocio real del proyecto: la canción se asigna al artista vinculado al usuario autenticado. Este comportamiento hace que la memoria explique

algo más útil que un simple listado de métodos, porque muestra cómo se protege la coherencia entre identidad y catálogo.

Otro fragmento clave se encuentra en `SongRepository`:

```
@Query("SELECT s FROM Song s WHERE s.active = true  
AND (s.releaseDate IS NULL OR s.releaseDate <= :today)")
```

La consulta anterior define qué canciones son públicas. No basta con que una canción exista; además debe estar activa y no tener una fecha futura de publicación. Este detalle convierte la base de datos en parte activa de la lógica de publicación y evita resolver todo en el frontend.

Por último, el backend protege el acceso al sistema de ficheros en `FileUploadResource`:

```
Path filePath = uploadPath.resolve(filename).normalize();  
  
if (!filePath.startsWith(uploadPath)) {  
    return ResponseEntity.badRequest().build();  
}
```

La normalización y la comprobación de prefijo bloquean rutas manipuladas que intenten salir de la carpeta autorizada. Se trata de una comprobación sencilla, pero técnicamente significativa, porque evita exponer archivos ajenos al directorio de subida.

La autenticación también merece una explicación explícita. En `DomainUserDetailsService` se decide cómo localizar al usuario que intenta entrar:

```
if (new EmailValidator().isValid(login, null)) {  
    return userRepository.findOneWithAuthoritiesByEmailIgnoreCase(login)  
    ...;  
}  
  
String lowercaseLogin = login.toLowerCase(Locale.ENGLISH);  
  
return userRepository.findOneWithAuthoritiesByLogin(lowercaseLogin) ...;
```

Este código permite autenticarse tanto con correo electrónico como con nombre de usuario. Además, normaliza el login a minúsculas para evitar inconsistencias por mayúsculas y minúsculas. No es una cuestión estética: afecta directamente a la robustez de la autenticación y a la experiencia del usuario.

La misma clase incorpora otra regla importante:

```
if (!user.isActivated()) {  
  
    throw new UserNotActivatedException("User " + lowercaseLogin + " was  
not activated");  
  
}
```

Con esta comprobación se impide que una cuenta no activada obtenga acceso, aunque sus credenciales sean correctas. La memoria debe subrayar este punto porque muestra que el sistema no solo valida contraseña, sino también el estado administrativo de la cuenta.

Una vez autenticado el usuario, `AuthenticateController` construye el JWT:

```
JwtClaimsSet.Builder builder = JwtClaimsSet.builder()  
  
    .issuedAt(now)  
  
    .expiresAt(validity)  
  
    .subject(authentication.getName())  
  
    .claim(AUTHORITIES_CLAIM, authorities);  
  
if (authentication.getPrincipal() instanceof UserWithId user) {  
  
    builder.claim(USER_ID_CLAIM, user.getId());  
  
}
```

Este fragmento es relevante porque explica qué contiene realmente el token. No solo guarda el nombre del usuario: también incluye las autoridades y el identificador interno. Gracias a ello, el backend puede autorizar peticiones futuras sin mantener sesión de servidor, y el sistema conserva el modelo stateless propio de JWT.

Otra pieza corta, pero muy expresiva, aparece en `SecurityConfiguration`:

```
.requestMatchers("/api/authenticate", "/api/register",  
"/api/activate").permitAll()  
  
.requestMatchers("/api/admin/**").hasAuthority(AuthoritiesConstants.ADMIN)  
  
.requestMatchers("/api/**").authenticated();
```

Estas reglas resumen la política general del backend. Algunas rutas deben ser públicas para poder iniciar sesión o registrar una cuenta; las rutas administrativas requieren un rol concreto; y el resto de la API exige autenticación. Explicarlo en la memoria es útil porque conecta el diseño de seguridad con el comportamiento visible del sistema.

4.2.3. Flujo backend de creación y publicación de canciones

Conviene explicar este flujo con más literalidad porque concentra varias decisiones relevantes del proyecto. Cuando un usuario autorizado crea una canción, el controlador REST recibe un `SongDTO`. Ese DTO no se persiste directamente. Primero se transforma en entidad mediante `SongMapper`, después se consulta el login autenticado y, finalmente, se localiza el artista asociado a ese login. Solo entonces se rellena el campo `artist` y se guarda el objeto.

Esta secuencia evita una vulnerabilidad muy típica en aplicaciones CRUD: que el cliente pueda falsificar la propiedad del contenido enviando un identificador de artista distinto. El backend no confía en el formulario para decidir la autoría principal, sino que deriva esa autoría del contexto de seguridad. En términos documentales, es una regla de negocio importante porque convierte una simple operación de guardado en un flujo de control de integridad.

La publicación pública sigue un principio parecido. La consulta no devuelve automáticamente toda canción persistida. Las consultas `findPublicSongs` y `findPublicSongsByAlbumId` exigen que la canción esté activa y que la fecha de lanzamiento no sea futura. Por tanto, el catálogo visible para el usuario final es un subconjunto filtrado del catálogo total. Esta decisión deja preparada la plataforma para trabajar con borradores, lanzamientos diferidos o desactivación temporal de contenidos.

El método `toggleActive` merece también una lectura funcional. No es solo un atajo de interfaz; representa una capacidad editorial para retirar o publicar un contenido sin borrarlo físicamente. En proyectos donde el historial o las relaciones importan, esta aproximación suele ser más razonable que eliminar filas de forma permanente.

4.2.4. Gestión de ficheros y streaming

La versión actual del proyecto ya permite reproducción real porque `FileUploadResource` expone rutas de carga para imágenes y audio. En ambos casos se valida el tipo MIME, se crea el directorio si todavía no existe y se genera un nombre aleatorio con `UUID` para reducir colisiones. El resultado devuelto al cliente contiene la URL pública y, en el caso del audio, también el nombre de fichero que luego utilizará el reproductor.

```
String filename = UUID.randomUUID().toString() + extension;

Path filePath = uploadPath.resolve(filename);

Files.copy(file.getInputStream(), filePath,
StandardCopyOption.REPLACE_EXISTING);
```

En el endpoint de streaming el objetivo ya no es guardar, sino servir el recurso correcto sin abrir la puerta a rutas arbitrarias. Por eso se normaliza el nombre recibido, se comprueba que el fichero siga dentro de la carpeta autorizada y solo después se construye un `UrlResource`. La respuesta anuncia `Content-Disposition: inline` y `Accept-Ranges: bytes`, pero esta última cabecera debe entenderse como base técnica para una evolución futura: la implementación actual sirve audio de forma correcta, aunque todavía no desarrolla por completo respuestas parciales [206](#).

Desde un punto de vista arquitectónico, esta solución separa bien tres responsabilidades: la base de datos conserva metadatos y relaciones, el sistema de ficheros almacena binarios y la API REST controla la exposición pública. En una memoria técnica, explicar esta triada es más valioso que copiar largas listas de propiedades, porque aclara cómo funciona de verdad la reproducción del proyecto.

4.3. Frontend Angular

El frontend está organizado por rutas y componentes standalone. Las pantallas de entidades se encuentran en `src/main/webapp/app/entities`, mientras que los servicios de autenticación, interceptores y utilidades comunes se ubican en `core` y `shared`. Esta separación replica en el cliente la modularidad del backend.

El inicio de sesión redirige a dashboards distintos según el rol del usuario. El administrador accede al panel de usuarios, catálogo, actividad y sistema; el editor accede a accesos rápidos de gestión musical; el usuario final accede a playlists, favoritos, historial y exploración del catálogo. La Figura 9 muestra una representación de la pantalla de administración basada en la interfaz implementada.

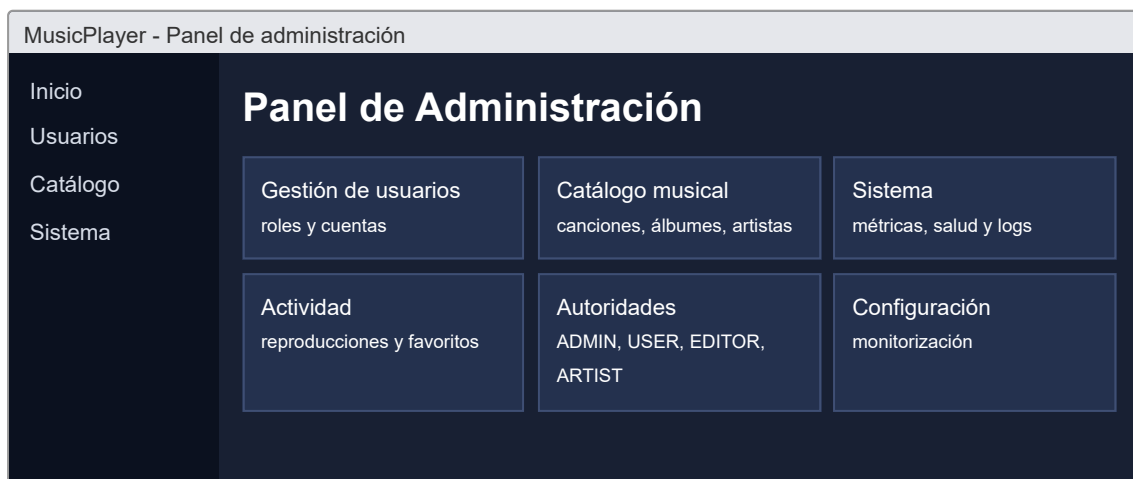


Figura 9. Vista funcional del panel de administración. Fuente: representación visual basada en la interfaz implementada.

La lista de canciones ofrece paginación, ordenación, acciones por rol y controles visuales de reproducción. Como se observa en la Figura 10, el diseño separa información musical, relaciones de álbum/género y acciones de mantenimiento. Los botones de creación, edición y eliminación solo se muestran a roles autorizados.

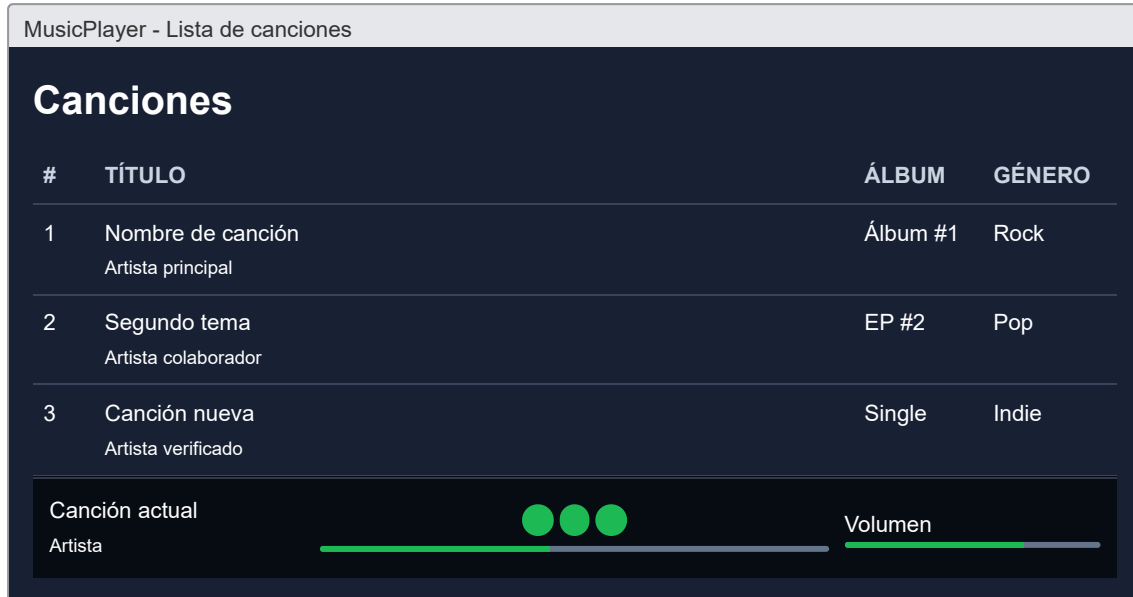


Figura 10. Vista funcional del listado de canciones y del reproductor. Fuente: representación visual basada en la interfaz implementada.

4.3.1. Clases frontend principales

La clase `login.ts` combina formulario reactivo, señales de error y redirección por rol. Tras autenticar al usuario, consulta la cuenta actual y decide si debe abrir el dashboard de administrador, editor o usuario. Esta lógica no sustituye a la autorización del backend, pero mejora la experiencia de navegación y muestra que el frontend participa en la adaptación de la interfaz al perfil autenticado.

```

this.accountService.identity().subscribe(account => {

  const roles = account?.authorities ?? [];

  if (roles.includes('ROLE_ADMIN')) {

    this.router.navigate(['/dashboard-admin']);

  } else if (roles.includes('ROLE_EDITOR')) {

    this.router.navigate(['/dashboard-editor']);

  } else {

    this.router.navigate(['/dashboard-user']);

  }

});

```

Este bloque muestra una decisión funcional clara: la misma autenticación conduce a experiencias distintas según el rol. En otras palabras, el frontend no cambia únicamente la decoración de la interfaz; reorganiza el punto de entrada del usuario en función de su responsabilidad dentro de la plataforma.

El servicio `login.service.ts` refuerza esta idea con una implementación deliberadamente pequeña:

```

login(credentials: Login): Observable<Account | null> {

  return this.authServerProvider.login(credentials)

    .pipe(mergeMap(() => this.accountService.identity(true)));

}

```

La secuencia es importante. Primero se solicita el token al backend y, a continuación, se recarga la identidad completa del usuario. Separar ambos pasos evita acoplar la interfaz a la estructura exacta de la respuesta JWT y permite que el cliente trabaje después con un objeto `Account` ya normalizado.

El servicio `player.service.ts` es una de las piezas más representativas del cliente porque no se limita a transportar datos: mantiene estado de reproducción mediante `signal`, calcula iconos y progreso con `computed` y sincroniza el objeto `Audio` del navegador usando `effect` y escuchadores de eventos. Cuando una canción se selecciona, construye la URL de reproducción; si `fileUrl` ya es una ruta completa la reutiliza, y si no lo es, llama al endpoint de streaming del backend.

```
const fileUrl = song.fileUrl ?? ''; this.audio.src =
fileUrl.startsWith('/') ? fileUrl

: /api/upload/stream/${encodeURIComponent(fileUrl)};
```

Este pequeño fragmento resume la integración entre cliente y servidor: el frontend reproduce audio, pero el backend conserva el control sobre cómo se expone físicamente el fichero. La Figura 11 sintetiza este comportamiento dentro del reproductor persistente.

Además, el propio servicio usa `effect` para sincronizar el volumen del navegador con el estado reactivo:

```
effect(() => {

  this.audio.volume = this.isMuted() ? 0 : this.volume() / 100;

});
```

Esta línea es breve, pero explica por qué el reproductor responde de manera inmediata a los controles de interfaz. La señal mantiene el estado lógico y el efecto traduce ese estado al objeto nativo `Audio`. Es un buen ejemplo de cómo Angular y la API del navegador cooperan dentro del proyecto.

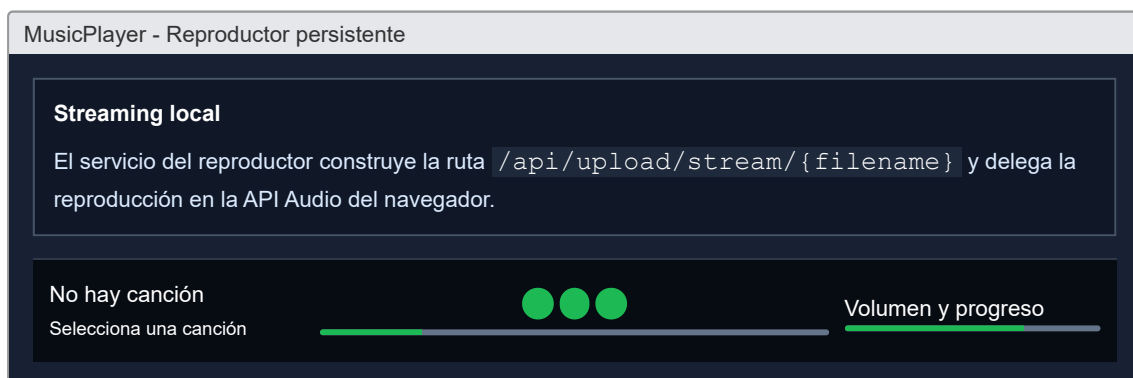


Figura 11. Vista funcional del reproductor persistente y del panel de letras. Fuente: representación visual basada en la interfaz implementada.

4.3.2. Organización de pantallas y navegación

El frontend no se ha concebido como una única pantalla con controles dispersos, sino como una colección de rutas con responsabilidades claras. Existen pantallas de administración, edición, exploración y detalle. Esta organización mejora la mantenibilidad porque cada vista concentra un objetivo: gestionar usuarios, editar catálogo, consultar listas o reproducir contenido. A la vez, facilita el control de acceso, ya que cada ruta puede protegerse con autoridades diferentes.

Los dashboards cumplen un papel especialmente importante. Funcionan como puntos de entrada diferenciados según el perfil autenticado. El administrador ve una visión orientada a control del sistema; el editor y el artista necesitan accesos rápidos al mantenimiento del catálogo; el usuario final necesita descubrir música, reproducirla y organizarla. Esta diferencia no es cosmética: expresa que el proyecto contempla varios actores con intereses distintos dentro de la misma plataforma.

También conviene destacar el uso de componentes de entidad generados por JHipster y después adaptados. Gracias a esa base, el proyecto dispone de formularios, listados, ordenación y paginación consistentes. La personalización posterior añade la lógica musical específica, de manera que el desarrollo no parte de cero, pero tampoco se queda en el código generado automáticamente.

4.3.3. Estado reactivo y sincronización del reproductor

El reproductor representa una pieza particularmente rica desde el punto de vista del frontend porque combina estado de aplicación y estado nativo del navegador. Las señales `currentSong`, `isPlaying`, `volume`, `progress` y `duration` modelan la situación lógica. En paralelo, el objeto `Audio` mantiene la reproducción real. El servicio actúa como puente entre ambos mundos.

El comportamiento de cola refuerza esta idea. Cuando se llama a `playSong`, el servicio no solo carga una canción: conserva la lista de reproducción activa y calcula el índice actual. A partir de ahí, los métodos `next` y `prev` pueden avanzar, retroceder o aplicar aleatorio sin que cada componente vuelva a implementar esas reglas. Esta centralización evita divergencias funcionales entre pantallas distintas.

Los escuchadores de eventos completan la sincronización. `timeupdate` actualiza progreso y tiempo reproducido, `loadedmetadata` inicializa la duración y `ended` decide si se repite la canción o se avanza a la siguiente. Este patrón resulta interesante para la memoria porque demuestra que el proyecto no se limita a consumir CRUD desde Angular, sino que también integra comportamiento multimedia real.

4.4. Persistencia y migraciones

El modelo de datos se define inicialmente en `music.jdl`. A partir de esa definición se generan entidades, repositorios, servicios, DTO, mappers, controladores y changelogs. La persistencia se realiza mediante Spring Data JPA e Hibernate. Liquibase versiona el esquema para que cada entorno pueda reproducir la misma estructura sin ejecutar scripts manuales.

El repositorio contiene 16 archivos de changelog Liquibase. Esto permite evolucionar la base de datos con trazabilidad: creación de entidades, restricciones, datos iniciales, nuevas autoridades y ajustes posteriores como la incorporación de `artist_id` a la tabla de canciones. La configuración de desarrollo apunta a MySQL en `localhost:3307/musicplayer`, mientras que producción usa MySQL en `localhost:3306/musicplayer`. Las pruebas de integración utilizan una configuración específica que permite ejecutar el backend de forma aislada.

La persistencia de ficheros también forma parte de esta sección porque el audio del proyecto no se queda en un valor abstracto de base de datos. La tabla `song` guarda la referencia `fileUrl`, pero el fichero real se almacena en disco y se publica por medio de Spring MVC. De este modo, la base de datos conserva metadatos y relaciones, mientras que el sistema de ficheros mantiene el contenido binario.

4.4.1. Migraciones Liquibase clave

Dentro de una memoria técnica, las migraciones no deberían presentarse como un detalle secundario. Son el mecanismo que garantiza que el modelo se despliegue igual en todos los entornos. En este proyecto, Liquibase crea tablas, restricciones, relaciones y datos iniciales, y también refleja la evolución del diseño cuando una necesidad aparece más tarde, como ocurre con la asociación entre canción y artista principal.

Tabla 9. Migraciones Liquibase clave

Changelog o grupo	Cambio aplicado	Justificación técnica
Creación inicial de entidades	Alta de tablas para género, artista, álbum, canción, playlist, reproducciones y favoritos.	Materializar el modelo musical definido en JDL con integridad referencial.
Relaciones muchos a muchos	Creación de tablas intermedias como <code>rel_song__artists</code> .	Representar colaboraciones musicales sin duplicar datos en canción o artista.
Autoridades personalizadas	Insertión de <code>ROLE_EDITOR</code> y <code>ROLE_ARTIST</code> .	Ampliar el esquema base de JHipster para separar consumo y edición de catálogo.

Changelog o grupo	Cambio aplicado	Justificación técnica
Ajuste <code>artist_id</code> en canción	Nueva columna y clave foránea desde <code>song</code> hacia <code>artist</code> .	Resolver la propiedad principal de la canción desde backend y simplificar consultas.
Datos y restricciones auxiliares	Índices, claves ajenas y valores por defecto.	Mantener consistencia y rendimiento mínimo en consultas frecuentes.

Este enfoque versionado resulta especialmente valioso en un entorno académico porque facilita repetir el despliegue, corregir errores sin tocar manualmente la base de datos y justificar por qué el modelo actual no surgió de una sola vez. El esquema ha evolucionado con el propio aprendizaje del proyecto y Liquibase deja ese rastro de manera explícita.

4.4.2. Correspondencia entre JDL, entidades y tablas

El archivo `music.jdl` actúa como descripción de alto nivel del dominio. A partir de él se generan clases Java, DTO, rutas Angular y tablas SQL, pero la versión actual del proyecto introduce personalizaciones posteriores. Esa combinación entre generación y ajuste manual es importante: el proyecto parte de una herramienta productiva, aunque la solución final ya no sea un simple resultado automático.

Tabla 10. Correspondencia entre modelo lógico y modelo físico

Nivel	Ejemplo en el proyecto	Papel en la solución
Modelo conceptual	Entidades Song, Album, Artist, Playlist y relaciones musicales.	Expresa necesidades de negocio y sirve de base para el diseño.
Definición JDL	<code>music.jdl</code> con entidades, campos y relaciones.	Permite generar estructura coherente de backend y frontend.
Modelo físico	Tablas <code>song</code> , <code>album</code> , <code>artist</code> , <code>playlist_song</code> , etc.	Persistencia real en MySQL con claves foráneas y restricciones.
Personalización posterior	Changelog que añade <code>artist_id</code> y controladores de subida/streaming.	Adapta la base generada a necesidades específicas de la versión actual del proyecto.

La correspondencia entre estos niveles es una de las razones por las que el proyecto resulta adecuado para una memoria técnica. Permite explicar no solo qué tablas existen, sino también cómo se ha llegado a ellas y qué cambios se han hecho para responder a requisitos que no quedaban completamente cubiertos por la generación inicial.

La Figura 2, basada en el esquema relacional del proyecto, permite relacionar de manera directa el diseño lógico con el funcionamiento real de la aplicación. En la parte superior del diagrama se encuentra el bloque de identidad y permisos, formado por `roles` y

`users`. Este bloque explica por qué la aplicación puede distinguir entre perfiles con capacidades distintas: administración, edición, gestión artística o consumo normal.

En el centro aparecen las tablas que conectan el uso cotidiano con el catálogo musical. `playlists` y `playlist_songs` explican cómo una persona usuaria crea listas y conserva el orden de las canciones. `plays` y `likes` representan dos acciones visibles en la experiencia de uso: reproducir contenido y marcarlo como favorito. No son simples adornos del modelo; son las piezas que permiten que el sistema recuerde actividad, preferencias y organización personal.

En la franja inferior del diagrama se concentra el catálogo principal, con `artists`, `songs`, `genres` y `albums`. La tabla `songs` ocupa una posición central porque es el elemento que finalmente consume el usuario. Una canción puede pertenecer a un álbum, clasificarse en un género y asociarse a uno o varios artistas. Desde el punto de vista funcional, esto hace posible mostrar fichas completas de contenido, agrupar canciones por publicación y permitir búsquedas o listados más coherentes.

La correspondencia entre JDL, entidades Java y tablas SQL se entiende mejor al leer este diagrama como una traducción gradual. Lo que en el modelo conceptual se formula como “un usuario crea una playlist”, en la base de datos se convierte en una fila en `playlists` enlazada con `users`. Lo que en la interfaz se percibe como “marcar me gusta”, se traduce en una relación persistida en `likes`. Y lo que en el reproductor se presenta como “reproducir una canción”, puede dejar rastro en `plays`. Esta cadena de correspondencias demuestra que la base de datos no es un apéndice técnico aislado, sino el soporte directo de la experiencia de uso.

Por eso, aunque el modelo se haya generado inicialmente a partir de `music.jdl` y después se haya personalizado con migraciones y código adicional, el resultado final mantiene coherencia entre diseño, implementación y comportamiento funcional. Esa coherencia es precisamente la que se espera evidenciar en una memoria técnica bien estructurada.

4.5. Seguridad e internacionalización

La seguridad combina Spring Security en el backend y guards/directivas en Angular. El backend es la fuente de verdad: aunque el cliente oculte botones, la autorización final se aplica en servidor. En el frontend, las rutas de creación y edición de canciones/álbumes requieren autoridades específicas, y la directiva `jhiHasAnyAuthority` permite ocultar acciones no permitidas.

La internacionalización se basa en archivos JSON por idioma en `src/main/webapp/i18n`. El idioma nativo es español y se incluye inglés como idioma

secundario. Esta decisión facilita que las etiquetas de formularios, mensajes de error y textos de navegación puedan ampliarse sin modificar los componentes.

4.5.1. Recorrido de una petición JWT

La autenticación JWT se entiende mejor si se describe el recorrido completo de una petición. El usuario se autentica enviando login y contraseña. El backend responde con un token firmado que contiene sujeto, autoridades e identificador de usuario. A partir de ese momento, el cliente adjunta el token en la cabecera `Authorization: Bearer ...`. Spring Security intercepta la petición, valida la firma y reconstruye el contexto autenticado antes de que el controlador de negocio llegue a ejecutarse.

Este recorrido justifica por qué la memoria insiste en distinguir autenticación y autorización. Autenticarse significa demostrar la identidad y obtener un token válido. Autorizar significa decidir, en cada petición concreta, si ese usuario puede o no realizar la acción solicitada. En la práctica, el proyecto usa ambas capas: primero crea el JWT y después limita determinadas rutas a administradores o a usuarios autenticados.

La ventaja de este modelo es que reduce dependencia de sesión de servidor y encaja bien con una SPA Angular. Su principal exigencia es mantener una configuración cuidada de expiración, renovación y protección del token en cliente. Aunque el prototipo ya funciona con este esquema, una evolución futura podría reforzar la persistencia segura del token y la gestión de caducidad en escenarios prolongados.

4.6. Despliegue y ejecución

Durante el desarrollo se ejecutan backend y frontend por separado: Spring Boot en el puerto 8080 y Angular en el puerto 4200. Para producción, Maven genera un paquete ejecutable que sirve la aplicación integrada y usa el perfil `prod`. La configuración productiva activa compresión, caché HTTP y una conexión MySQL con HikariCP.

El despliegue no requiere licencias de pago. Todas las tecnologías principales son de código abierto o gratuitas para el contexto del proyecto. Esto favorece que el prototipo pueda instalarse en un entorno educativo o de pruebas con coste reducido.

4.7. Planificación temporal

La planificación se organizó en fases porque el proyecto combina aprendizaje tecnológico, generación inicial, personalización del dominio, desarrollo visual, pruebas y documentación. En una aplicación generada con JHipster existe la tentación de considerar que la fase inicial resuelve la mayor parte del trabajo. Sin embargo, la experiencia muestra que la generación solo proporciona una base: el tiempo real se

invierte en comprenderla, ajustarla, validar los permisos, adaptar la interfaz y documentar las decisiones.

La Tabla 11 recoge una planificación temporal razonada. Las semanas no deben interpretarse como bloques totalmente aislados; varias tareas se solapan, especialmente pruebas y documentación. Aun así, la división permite entender el avance del proyecto y justificar el orden de ejecución.

Tabla 11. Planificación temporal

Fase	Duración estimada	Actividades principales	Resultado
Investigación y definición	2 semanas	Análisis de plataformas musicales, selección de tecnologías y definición del alcance.	Objetivos, requisitos y enfoque técnico.
Modelado del dominio	1 semana	Diseño JDL, entidades, relaciones y validaciones principales.	Modelo de datos inicial y generación de entidades.
Backend y seguridad	3 semanas	Configuración JWT, roles, recursos REST, DTO, mappers y migraciones.	API funcional y persistencia reproducible.
Frontend y experiencia de usuario	3 semanas	Dashboards, listados, formularios, navegación, reproductor visual y letras.	Interfaz navegable por roles.
Pruebas y correcciones	2 semanas	Pruebas unitarias, integración, validaciones manuales y revisión de errores.	Plan de pruebas y comportamiento validado.
Documentación final	2 semanas	Redacción de memoria, anexos, diagramas, índices, bibliografía y revisión formal.	Memoria técnica entregable en PDF/Word.

La fase con mayor incertidumbre fue la de seguridad. Aunque JHipster genera una configuración inicial correcta, entender la cadena de filtros, las autoridades y la interacción entre backend y guards de Angular requiere una lectura cuidadosa. Esta dificultad no se percibe al mirar solo las clases generadas, pero sí aparece cuando se intenta explicar el sistema con precisión.

4.8. Estimación económica

La estimación económica se presenta con finalidad académica. El proyecto se ha desarrollado con herramientas gratuitas y equipos ya disponibles, por lo que no se ha producido un coste directo de licencias. Aun así, para valorar el esfuerzo real se puede estimar el coste de personal y de recursos si el prototipo se encargara como desarrollo profesional.

Tabla 12. Estimación económica

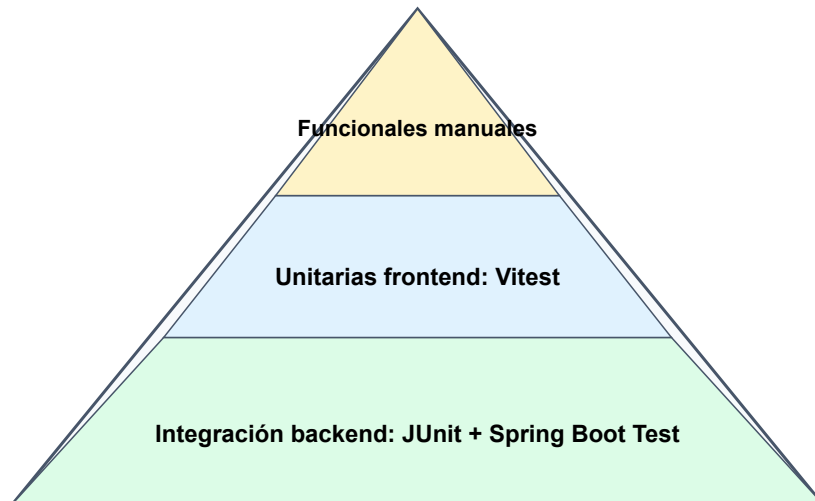
Concepto	Criterio	Coste estimado
Equipo de desarrollo	Ordenador personal ya disponible.	0 € de coste adicional.
Software base	JDK, Node.js, Spring Boot, Angular, JHipster, MySQL Community, Liquibase y VS Code/IDE comunitario.	0 € en licencias.
Infraestructura local	Uso de entorno local y Docker para servicios auxiliares.	0 € en fase de desarrollo.
Trabajo de análisis y desarrollo	Aproximadamente 13 semanas de trabajo académico parcial.	Coste imputable al esfuerzo formativo.
Despliegue futuro	Servidor VPS básico, dominio y certificado TLS.	Pendiente de decisión; no incluido en el prototipo.

La principal inversión del proyecto no ha sido económica, sino de aprendizaje y tiempo. Esta observación es relevante porque una memoria técnica no debe limitarse a enumerar herramientas: debe dejar constancia de qué decisiones permitieron reducir coste, cuáles aumentaron la complejidad y qué partes exigirían inversión adicional para producción.

5. Pruebas realizadas y validación

5.1. Estrategia de pruebas

La validación se organiza en niveles. Las pruebas unitarias del frontend verifican servicios, formularios y componentes; las pruebas de integración del backend arrancan el contexto Spring y validan recursos REST; las pruebas funcionales manuales recorren los flujos completos desde el navegador. La Figura 12 resume esta estrategia en forma de pirámide.



Mayor automatización en la base; validación de experiencia completa en la parte superior.

Figura 12. Pirámide de pruebas del proyecto. Fuente: elaboración propia.

Tabla 13. Estrategia de pruebas

Tipo	Herramienta	Alcance	Evidencia
Unitarias frontend	Vitest / Angular TestBed	Servicios, componentes, formularios y utilidades.	116 archivos <code>.spec.ts</code> .
Integración backend	JUnit 5 + Spring Boot Test	Controladores REST, repositorios, mappers y seguridad.	87 archivos Java de prueba.
Funcionales manuales	Navegador y Postman	Login, roles, CRUD, playlists, favoritos y letras.	Plan de pruebas documentado.
Regresión visual	Revisión manual en navegador	Coherencia de dashboards, listados, formularios y reproductor.	Comparación tras cambios de SCSS/HTML.

5.2. Casos de prueba funcionales

La Tabla 14 resume los casos funcionales más representativos. Se han seleccionado los que cubren mayor riesgo: seguridad, catálogo, permisos, playlists y dependencia externa.

Tabla 14. Casos de prueba funcionales

ID	Módulo	Pasos resumidos	Resultado esperado
AUTH-01	Autenticación	Entrar en <code>/login</code> , introducir credenciales válidas y enviar.	Se obtiene JWT y se redirige al dashboard según rol.
AUTH-02	Autenticación	Introducir credenciales inválidas.	Se muestra error y no se crea sesión.
AUTH-03	Autorización	Acceder a una ruta de administración con usuario sin permisos.	El sistema deniega el acceso.

ID	Módulo	Pasos resumidos	Resultado esperado
CAT-01	Canciones	Listar canciones autenticado.	Se muestran registros paginados y cabecera de total.
CAT-02	Canciones	Crear canción como editor con campos válidos.	La API devuelve creación correcta y la canción aparece en listado.
CAT-03	Canciones	Intentar crear canción sin título.	El formulario o backend rechaza la operación.
PL-01	Playlists	Crear playlist y asociar una canción mediante PlaylistSong.	La relación queda persistida con posición.
LIKE-01	Favoritos	Crear un Like entre usuario y canción.	La canción aparece como favorita del usuario.
LYR-01	Letras	Abrir panel de letras con canción seleccionada.	Se muestra letra o mensaje de no disponible sin romper la interfaz.
I18N-01	Internacionalización	Cambiar idioma de la interfaz.	Las etiquetas traducibles cambian entre español e inglés.

5.3. Validación técnica

La validación técnica se apoya en la compilación Maven, las pruebas generadas por JHipster, las especificaciones Angular y la comprobación manual de rutas. La existencia de pruebas no garantiza por sí sola la ausencia de defectos, pero proporciona una red de seguridad para cambios en entidades, DTO, controladores y formularios. La memoria no incluye listados completos de código porque no aportan valor documental; los fragmentos técnicos quedan limitados a comandos de instalación en anexos.

En términos de aceptación, se considera que una funcionalidad supera la validación cuando cumple tres condiciones: la interfaz permite ejecutar el flujo, el backend responde con el estado HTTP esperado y la base de datos conserva la relación correcta entre entidades. Para el caso del audio, además, la reproducción debe resolver correctamente la ruta del fichero y mantener sincronizados el estado visual y el estado real del objeto `Audio`.

5.3.1. Evidencias automáticas disponibles

La versión utilizada aporta una base sólida de validación automática gracias a la estructura estándar de JHipster y a las pruebas añadidas en frontend. Esta circunstancia es relevante porque permite comprobar que la aplicación no solo compila, sino que mantiene contratos internos entre capas. Un fallo en un mapper, en una ruta REST o en un formulario suele reflejarse pronto en las suites existentes.

Tabla 15. Evidencias de validación técnica

Evidencia	Qué demuestra	Límite de la evidencia
Pruebas JUnit de recursos y servicios	Que controladores, mappers y repositorios mantienen contratos mínimos esperados.	No sustituyen la validación manual de permisos finos ni de experiencia de usuario.
Especificaciones Angular	Que servicios, componentes y formularios conservan comportamiento básico.	No cubren por sí solas navegación completa de extremo a extremo.
Compilación Maven/Angular	Que dependencias, tipado y configuración principal son coherentes.	Compilar no garantiza que todos los flujos de negocio estén bien resueltos.
Comprobación manual de login, catálogo y reproducción	Que los flujos principales funcionan de manera integrada.	Depende de la disciplina de ejecución y de la cobertura de escenarios elegidos.

Desde el punto de vista de la memoria, esta tabla permite demostrar que la validación no se ha reducido a “se ha probado y funciona”. Se identifican evidencias concretas, su alcance y su límite. Esa precisión es importante en un documento académico porque refleja criterio técnico y no solo optimismo sobre el resultado.

5.3.2. Riesgos no cubiertos completamente

Ningún proyecto académico cubre todos los riesgos posibles, y conviene reconocerlo de forma explícita. En esta versión del proyecto siguen existiendo áreas que merecerían más validación si la aplicación evolucionara a un entorno productivo: concurrencia en subida de ficheros, límites de almacenamiento, comportamiento ante audios corruptos, soporte real de rangos HTTP, endurecimiento de permisos por entidad y automatización end-to-end de la navegación por roles.

Nombrar estos riesgos no debilita la memoria; al contrario, la hace más creíble. Un documento técnico sólido no solo enumera aciertos, sino que también marca el perímetro de lo ya resuelto y de lo todavía pendiente. Esa delimitación ayuda al tutor a evaluar el proyecto con criterios realistas.

5.4. Resultados por módulo

La Tabla 16 resume el estado de validación por módulo. Esta tabla sustituye a los listados extensos de funciones y código por una visión de calidad: qué se ha comprobado, qué resultado se espera y qué riesgo residual queda. De esta forma, la memoria mantiene un enfoque técnico sin convertirse en documentación automática.

Tabla 16. Resultados por módulo

Módulo	Validación realizada	Resultado	Riesgo residual
--------	----------------------	-----------	-----------------

Módulo	Validación realizada	Resultado	Riesgo residual
Autenticación	Login correcto, login incorrecto, persistencia de token y redirección por rol.	Flujo principal correcto.	Revisar expiración y renovación de token en sesiones largas.
Administración	Acceso a panel, usuarios, autoridades y endpoints de gestión.	Acceso restringido a rol administrador.	Añadir auditoría visible de cambios administrativos.
Catálogo musical	CRUD de canciones, artistas, álbumes y géneros con validaciones.	Operaciones disponibles para roles autorizados.	Refinar permisos backend por entidad si se separan editor y artista.
Playlists	Creación de listas y relación ordenada con canciones.	Modelo preparado para ordenar contenido.	Implementar reglas estrictas de propiedad en todas las operaciones.
Favoritos	Relación Like entre usuario y canción.	Persistencia simple y extensible.	Evitar duplicados con restricción única usuario-canción.
Reproducciones	Registro de Play asociado a usuario y canción.	Base para historial y estadísticas.	Añadir lógica automática al iniciar reproducción real.
Audio y reproducción	Construcción de la URL de stream, carga del fichero y actualización del estado del reproductor.	Integración funcional entre frontend y backend.	Añadir control de rangos, métricas y límites de almacenamiento.
Internacionalización	Existencia de recursos en español e inglés.	Base bilingüe disponible.	Completar traducciones de textos personalizados nuevos.

La validación también ha servido para identificar límites. En la versión actual del proyecto analizada ya existe una primera solución de carga y streaming mediante `FileUploadResource` y la carpeta `uploads`, pero una plataforma final debería completar esa base con cuotas, análisis de ficheros, borrado seguro, soporte real de rangos parciales y almacenamiento externo si el volumen crece. Del mismo modo, la entidad Like permite favoritos, pero conviene reforzar las restricciones de unicidad para impedir duplicados por usuario y canción.

5.5. Matriz de trazabilidad

La trazabilidad relaciona requisitos, diseño, implementación y pruebas. Su utilidad es demostrar que las secciones de la memoria no son independientes, sino partes conectadas de una misma solución. La Tabla 17 recoge una matriz resumida.

Tabla 17. Matriz de trazabilidad

Requisito	Elemento de diseño	Implementación	Prueba asociada
-----------	--------------------	----------------	-----------------

Requisito	Elemento de diseño	Implementación	Prueba asociada
RF-01 Autenticación JWT	Actividad de inicio de sesión y arquitectura cliente-servidor.	<code>AuthenticateController</code> , <code>Spring Security</code> y servicio de login Angular.	AUTH-01, AUTH-02, AUTH-03.
RF-03 Gestión de canciones	Entidad Song, relaciones con Album, Genre y Artist.	<code>SongResource</code> , <code>SongService</code> , rutas <code>/songs</code> .	CAT-01, CAT-02, CAT-03.
RF-04 Gestión de álbumes	Entidad Album y enumeración AlbumType.	<code>AlbumResource</code> , formulario Angular de álbum.	Pruebas CRUD de entidad y revisión manual.
RF-07 Playlists	Entidades Playlist y PlaylistSong.	<code>PlaylistResource</code> y <code>PlaylistSongResource</code> .	PL-01.
RF-09 Favoritos	Entidad Like vinculada a User y Song.	<code>LikeResource</code> y vistas de favoritos.	LIKE-01.
RF-10 Dashboards por rol	Casos de uso de administrador, editor/artista y usuario.	Rutas <code>/dashboard-admin</code> , <code>/dashboard-editor</code> , <code>/dashboard-user</code> .	Validación manual de navegación por rol.
RF-12 Reproducción y letras almacenadas	Reproductor persistente, entidad Song y flujo de streaming.	<code>player.service.ts</code> , <code>FileUploadResource</code> y campo <code>lyrics</code> .	Validación manual de reproducción y consulta de detalle.
RNF-04 Migraciones	Modelo relacional versionado.	Changelogs Liquibase.	Arranque de backend y pruebas de integración.

5.6. Criterios de aceptación final

Para considerar el prototipo apto como proyecto intermodular, se establecen criterios de aceptación ligados a los requisitos. Primero, la aplicación debe arrancar de forma reproducible con los comandos documentados. Segundo, un usuario debe poder autenticarse y acceder a vistas protegidas. Tercero, las entidades principales deben poder gestionarse desde API y desde interfaz. Cuarto, las relaciones entre entidades deben conservar integridad referencial. Quinto, la documentación debe permitir entender arquitectura, diseño, pruebas y despliegue sin leer directamente el código fuente.

La versión de memoria generada cumple además criterios formales: portada, resumen, abstract, índices, numeración de figuras y tablas, bibliografía en formato APA 7, anexos y lenguaje impersonal. Esta revisión formal responde directamente a la observación recibida por correo, donde se indicaba que la entrega anterior parecía documentación generada automáticamente y carecía de explicaciones.

6. Conclusiones y mejoras futuras

6.1. Conclusiones

El proyecto demuestra que es posible construir una plataforma musical autogestionada usando una pila tecnológica moderna y ampliamente utilizada. La combinación de JHipster, Spring Boot y Angular ha permitido avanzar con rapidez sin renunciar a una estructura profesional: capas separadas, seguridad JWT, migraciones Liquibase, DTO, mappers y pruebas.

Desde el punto de vista del aprendizaje, el proyecto ha exigido comprender cómo se conectan decisiones de dominio con decisiones de infraestructura. Una entidad aparentemente simple, como PlaylistSong, resuelve una necesidad real de negocio: mantener el orden de las canciones dentro de una playlist. De forma similar, la introducción de roles personalizados permite adaptar una base genérica de JHipster a un contexto musical con administración, edición y consumo.

La principal limitación actual es que algunas funcionalidades propias de una plataforma comercial de streaming se encuentran implementadas en una primera versión técnica, pero no alcanzan todavía un nivel productivo completo. En particular, la gestión de archivos ya dispone de carga y streaming local en backend, aunque todavía requiere endurecimiento operativo y de seguridad para un entorno real con mayor volumen.

6.2. Mejoras futuras

- Reforzar el servicio backend de subida de audio e imágenes con límites de tamaño, antivirus, trazabilidad y borrado seguro.
- Añadir streaming con soporte real de rangos HTTP y respuestas parciales para permitir saltos dentro del audio con mayor eficiencia.
- Desarrollar un servicio de búsqueda por título, artista, álbum y género con filtros combinados.
- Completar la experiencia social con seguimiento de artistas y perfiles extendidos si se decide activar esas entidades en JHipster.
- Incorporar pruebas end-to-end con Playwright o Cypress para automatizar flujos completos de navegador.
- Mejorar el panel de letras con caché local, timeout y sincronización temporal cuando exista información disponible.
- Preparar un despliegue Docker completo con variables de entorno, secretos externos y configuración TLS.

- Revisar accesibilidad: navegación por teclado, contraste, etiquetas ARIA y mensajes de validación.

7. Bibliografía y recursos consultados

- Angular. (s. f.). *Angular documentation*. Recuperado el 12 de mayo de 2026, de <https://angular.dev/>
- Bootstrap. (s. f.). *Bootstrap documentation 5.3*. Recuperado el 12 de mayo de 2026, de <https://getbootstrap.com/docs/5.3/>
- JHipster. (s. f.). *JHipster documentation archive v9.0.0*. Recuperado el 12 de mayo de 2026, de <https://www.jhipster.tech/documentation-archive/v9.0.0/>
- Liquibase. (s. f.). *Liquibase documentation*. Recuperado el 12 de mayo de 2026, de <https://docs.liquibase.com/>
- MapStruct. (s. f.). *MapStruct reference guide*. Recuperado el 12 de mayo de 2026, de <https://mapstruct.org/documentation/stable/reference/html/>
- MySQL. (s. f.). *MySQL documentation*. Recuperado el 12 de mayo de 2026, de <https://dev.mysql.com/doc/>
- OWASP Foundation. (2021). *OWASP Top Ten*. <https://owasp.org/www-project-top-ten/>
- Spring. (s. f.). *Spring Boot reference documentation*. Recuperado el 12 de mayo de 2026, de <https://docs.spring.io/spring-boot/>
- Spring Security. (s. f.). *Spring Security reference*. Recuperado el 12 de mayo de 2026, de <https://docs.spring.io/spring-security/reference/>
- Spring Data. (s. f.). *Spring Data JPA reference documentation*. Recuperado el 12 de mayo de 2026, de <https://docs.spring.io/spring-data/jpa/reference/>

8. Anexos

8.1. Anexo A: Manual de instalación

8.1.1. Requisitos previos

Para ejecutar el proyecto se requiere JDK 21, Node.js compatible con el proyecto, Maven o el wrapper incluido, Git y una base de datos MySQL si se usa la configuración local actual. El repositorio incluye wrappers `mvnw` y `npmw`, por lo que no es imprescindible instalar Maven o npm globalmente si se trabaja con esos wrappers.

8.1.2. Puesta en marcha en desarrollo

```
git clone <url-del-repositorio>

cd music-player

./npmw install

docker compose -f src/main/docker/mysql.yml up --wait

./mvnw -Dskip.installnodenpm -Dskip.npm

./npmw run start
```

Con esta configuración, el backend queda disponible en `http://localhost:8080` y el frontend Angular en `http://localhost:4200`. La configuración de desarrollo del repositorio apunta a MySQL en `localhost:3307/musicplayer`. Los ficheros subidos quedan, por defecto, en la carpeta `uploads` definida por la propiedad `app.upload.dir`.

8.1.3. Construcción de producción

```
./mvnw -Pprod clean verify

java -jar target/*.jar
```

El perfil de producción usa MySQL en `localhost:3306/musicplayer` según la configuración actual. En un despliegue real se recomienda externalizar credenciales y secretos mediante variables de entorno o un gestor de secretos.

8.2. Anexo B: Manual de usuario

8.2.1. Inicio de sesión

El usuario accede a la pantalla de login, introduce sus credenciales y pulsa el botón de inicio de sesión. Si los datos son correctos, el sistema obtiene el perfil y lo redirige al panel que corresponde a su rol. Si son incorrectos, permanece en la pantalla de login y muestra un mensaje de error.

8.2.2. Gestión de catálogo

Los usuarios con rol autorizado pueden crear canciones, álbumes, artistas y géneros desde las rutas de entidad. Cada formulario solicita los campos obligatorios y muestra mensajes de validación cuando falta información requerida. Tras guardar, el usuario regresa al listado y puede consultar el detalle.

8.2.3. Playlists, favoritos y reproducciones

El usuario final puede navegar al catálogo, consultar canciones, crear playlists y asociar canciones a listas mediante la entidad PlaylistSong. También puede registrar favoritos y consultar el historial de reproducciones si dispone de datos asociados.

8.2.4. Reproducción y letras almacenadas

Desde la barra de reproducción el usuario puede iniciar, pausar, avanzar o retroceder dentro de la cola actual. El cliente solicita el audio al endpoint `/api/upload/stream/{filename}` cuando la canción se ha registrado con un nombre de fichero local. Si la canción dispone de letra almacenada en el campo `lyrics`, esta puede consultarse desde las vistas de detalle o edición sin depender de servicios externos.

8.3. Anexo C: Recursos REST principales

La Tabla 18 resume los recursos REST de negocio. No se incluyen todos los métodos ni cuerpos JSON para evitar convertir la memoria en una referencia automática de endpoints; el objetivo es ubicar el contrato principal de la API.

Tabla 18. Recursos REST principales

Recurso	Ruta base	Responsabilidad
AuthenticateController	<code>/api/authenticate</code>	Autenticación y emisión de JWT.
AccountResource	<code>/api/account</code>	Cuenta actual, registro, activación y contraseña.
UserResource	<code>/api/admin/users</code>	Administración de usuarios.
AuthorityResource	<code>/api/authorities</code>	Gestión de autoridades del sistema.
SongResource	<code>/api/songs</code>	CRUD de canciones y relaciones con álbum, género y artistas.
AlbumResource	<code>/api/albums</code>	CRUD de álbumes y tipo de publicación.
ArtistResource	<code>/api/artists</code>	CRUD de artistas y verificación.
GenreResource	<code>/api/genres</code>	CRUD de géneros musicales.
PlaylistResource	<code>/api/playlists</code>	CRUD de playlists de usuario.
PlaylistSongResource	<code>/api/playlist-songs</code>	Asociación ordenada entre playlists y canciones.
PlayResource	<code>/api/plays</code>	Historial de reproducciones.
LikeResource	<code>/api/likes</code>	Favoritos de canciones por usuario.

8.4. Anexo D: Glosario

Autoridad: permiso asociado a un usuario en Spring Security, por ejemplo `ROLE_ADMIN`.

Changelog: archivo Liquibase que describe un cambio versionado de base de datos.

DTO: objeto de transferencia que desacopla la API de las entidades persistentes.

Entidad: clase de dominio persistida en base de datos mediante JPA.

Guard: mecanismo Angular que decide si una ruta puede activarse según autenticación y roles.

Mapper: componente MapStruct que transforma entidades en DTO y viceversa.

SPA: aplicación de una sola página que cambia de vistas en el navegador sin recargar toda la página.

8.5. Anexo E: Diccionario de datos

El diccionario de datos complementa el modelo entidad-relación. Mientras el diagrama permite comprender relaciones, el diccionario ayuda a interpretar el significado de los campos principales y las restricciones que afectan al comportamiento de la aplicación. La Tabla 19 resume los atributos más relevantes.

Tabla 19. Diccionario de datos

Entidad	Campo	Tipo	Restricción / significado
Genre	id	Long	Clave primaria generada por la base de datos.
Genre	name	String	Nombre único del género; obligatorio y limitado a 50 caracteres.
Artist	id	Long	Clave primaria del artista.
Artist	name	String	Nombre artístico; obligatorio y limitado a 100 caracteres.
Artist	bio	TextBlob	Biografía o descripción extendida.
Artist	image	String	URL o ruta de imagen representativa.
Artist	country	String	Código de país de dos caracteres.
Artist	verified	Boolean	Indica si el artista ha sido verificado por la plataforma.
Album	title	String	Título del álbum; obligatorio y limitado a 150 caracteres.
Album	coverImage	String	Imagen de portada enlazada por URL o ruta.
Album	releaseDate	LocalDate	Fecha de publicación.
Album	albumType	Enum	Clasificación: ALBUM, SINGLE, EP o PODCAST_SERIES.

Entidad	Campo	Tipo	Restricción / significado
Song	title	String	Título de la canción; obligatorio y limitado a 150 caracteres.
Song	duration	Integer	Duración en segundos.
Song	fileUrl	String	Referencia al archivo de audio; obligatoria en el modelo actual.
Song	coverImage	String	Imagen asociada a la canción.
Song	lyrics	TextBlob	Letra almacenada si se dispone de ella.
Song	releaseDate	LocalDate	Fecha de lanzamiento de la canción.
Playlist	name	String	Nombre de la lista; obligatorio y limitado a 100 caracteres.
Playlist	description	TextBlob	Descripción opcional de la playlist.
Playlist	isPublic	Boolean	Determina si la lista es visible para otros usuarios.
Playlist	coverImage	String	Imagen de portada de la playlist.
PlaylistSong	position	Integer	Orden de la canción dentro de la playlist.
PlaylistSong	addedAt	Instant	Fecha y hora de incorporación de la canción.
Play	playedAt	Instant	Momento en que se registró la reproducción.
Play	durationListened	Integer	Segundos escuchados en la reproducción.
Like	createdAt	Instant	Fecha y hora en que se marcó la canción como favorita.

El diccionario permite detectar mejoras de integridad. Por ejemplo, en una versión posterior convendría añadir restricciones únicas compuestas en favoritos y playlists para impedir duplicidades no deseadas. También sería recomendable normalizar el almacenamiento de archivos para que `fileUrl` apunte a recursos controlados por el sistema y no a valores introducidos manualmente.

8.6. Anexo F: Guía de mantenimiento

El mantenimiento del proyecto debe realizarse con cuidado porque JHipster combina código generado y código personalizado. La recomendación general es no modificar una entidad o relación únicamente desde la base de datos; cualquier cambio estructural debe reflejarse en el JDL, en los DTO, en los mappers, en las migraciones y en las pruebas afectadas.

Tabla 20. Operaciones de mantenimiento

Operación	Procedimiento recomendado	Comprobación posterior
Añadir un campo a una entidad	Actualizar JDL o entidad, generar/crear changelog Liquibase, ajustar DTO, mapper y formulario Angular.	Compilar backend, revisar formulario y ejecutar pruebas de entidad.

Operación	Procedimiento recomendado	Comprobación posterior
Cambiar una relación	Analizar cardinalidad, impacto en datos existentes y consultas; crear migración reversible si es posible.	Validar integridad referencial y listados paginados.
Añadir un rol	Registrar autoridad en datos iniciales o changelog, actualizar constantes backend/frontend y proteger rutas.	Probar login y navegación con usuario que tenga el nuevo rol.
Modificar textos de interfaz	Actualizar archivos i18n en español e inglés.	Cambiar idioma en la aplicación y revisar que no falten claves.
Actualizar dependencias frontend	Revisar changelog de Angular/librerías, ejecutar instalación controlada y pruebas.	Ejecutar build y specs afectadas.
Actualizar Spring/JHipster	Crear rama separada, revisar breaking changes y aplicar migraciones gradualmente.	Compilar, ejecutar pruebas backend y verificar seguridad.
Preparar despliegue productivo	Externalizar secretos, configurar MySQL, revisar CORS, activar TLS y definir backups.	Prueba de arranque, login, operaciones CRUD y restauración de backup.